Estructura de Datos

Cristian Narváez Guillén

2026-03-01

Tabla de contenidos

Preface 5

1 Summary 6

2 Introducción 7
- 2.1 Introducción a las Estructuras de Datos y Algoritmos . . . 8
  - 2.1.1 ¿Qué son las Estructuras de Datos? . . . 8
  - 2.1.2 Abstracción y Tipos Abstractos de Datos (TAD) . . . 8
  - 2.1.3 Algoritmos y Eficiencia . . . 9
  - 2.1.4 El Camino hacia la Solución de Problemas . . . 9

3 Capítulo 1: Estructuras Estáticas 10
- 3.1 Introducción . . . 10
- 3.2 Arreglos: La Caja de Herramientas Estática . . . 10
  - 3.2.1 La Memoria como un Gran Casillero . . . 10
  - 3.2.2 El Lado Oscuro: Inserciones y Eliminaciones . . . 12
- 3.3 Archivos: Persistencia más allá de la RAM . . . 12
- 3.4 Matrices y Arreglos Multidimensionales . . . 12
  - 3.4.1 Mapeo de 2D a Memoria Lineal . . . 13
- 3.5 Cadenas de Caracteres (Strings) . . . 13
- 3.6 Metodologías Activas . . . 14
  - 3.6.1 Autoevaluación . . . 14
  - 3.6.2 Resolución de Problemas . . . 14

4 Estructuras Dinámicas 15

5 Introducción a las Estructuras Dinámicas en R 16

6 Listas Enlazadas 17
- 6.1 Listas (Generales en R) . . . 17
- 6.2 Listas Simples (Singly Linked Lists) . . . 18
- 6.3 Listas Dobles (Doubly Linked Lists) . . . 21
- 6.4 Listas Circulares (Circular Linked Lists) . . . 25

7 Pilas (Stacks) 29

8 Colas (Queues) **32**

9 Conclusión **35**

10 Algoritmos de Ordenación **36**

11 Algoritmos de Ordenación **37**
11.1 Burbuja (Bubble Sort) . . . 37
11.1.1 Concepto . . . 37
11.1.2 Implementación en R . . . 37
11.1.3 Explicación del Código . . . 38
11.1.4 Complejidad . . . 39
11.1.5 Ejemplo . . . 39
11.2 Inserción (Insertion Sort) . . . 40
11.2.1 Concepto . . . 40
11.2.2 Implementación en R . . . 41
11.2.3 Explicación del Código . . . 41
11.2.4 Complejidad . . . 42
11.2.5 Ejemplo . . . 42
11.3 Mergesort . . . 43
11.3.1 Concepto . . . 43
11.3.2 Implementación en R . . . 43
11.3.3 Explicación del Código . . . 44
11.3.4 Complejidad . . . 45
11.3.5 Ejemplo . . . 46
11.4 Quicksort . . . 46
11.4.1 Concepto . . . 46
11.4.2 Implementación en R . . . 47
11.4.3 Explicación del Código . . . 47
11.4.4 Complejidad . . . 48
11.4.5 Ejemplo . . . 48
11.5 Conclusión . . . 49

12 Algoritmos de Búsqueda **51**
12.1 Búsqueda Secuencial (Linear Search) . . . 51
12.1.1 ¿Cómo funciona? . . . 51
12.1.2 Implementación en R . . . 52
12.1.3 Ejemplos de Uso . . . 52
12.1.4 Eficiencia y Casos de Uso . . . 54
12.2 Búsqueda Binaria (Binary Search) . . . 54
12.2.1 ¿Cómo funciona? . . . 54
12.2.2 Implementación en R . . . 55
12.2.3 Ejemplos de Uso . . . 56

12.2.4 Eficiencia y Casos de Uso . . . 58
12.3 Comparación y Cuándo Usar Cada Uno . . . 58
12.4 Conclusión . . . 59

13 Estructuras No Lineales **60**
13.1 Grafos: Tejiendo Redes de Relaciones . . . 60
13.1.1 Tipos de Grafos . . . 60
13.1.2 Grafos en R: El Paquete `igraph` . . . 61
13.1.3 Propiedades y Análisis de Grafos . . . 62
13.2 Árboles Binarios: Jerarquías con Ramificaciones Limitadas . . . 63
13.2.1 Implementando Árboles Binarios en R . . . 63
13.2.2 Operaciones Comunes en Árboles Binarios (Conceptos) . . . 64
13.3 Conclusión . . . 65

14 Estructuras Avanzadas **66**
14.1 Árboles AVL . . . 66
14.1.1 ¿Por qué son importantes? . . . 66
14.1.2 Árboles AVL y R . . . 67
14.2 Árboles B . . . 68
14.2.1 Características Clave . . . 68
14.2.2 ¿Por qué son importantes? . . . 68
14.2.3 Árboles B y R . . . 69
14.3 Fibonacci Heap . . . 70
14.3.1 Operaciones y Complejidad Amortizada . . . 70
14.3.2 ¿Por qué son importantes? . . . 71
14.3.3 Fibonacci Heaps y R . . . 71
14.3.4 Conclusión . . . 72

15 Arboles_B_R **74**

References **75**

Preface

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

1 Summary

In summary, this book has no content whatsoever.

2 Introducción

La Universidad Nacional de Loja (UNL), con un legado histórico que se remonta a 1859, se consolida como una institución de educación superior de vital importancia para el desarrollo académico, tecnológico y social de la región y del país.

A través de facultades como la de la Energía, las Industrias y los Recursos Naturales No Renovables, y carreras como Computación, la UNL demuestra un profundo compromiso con la innovación y la excelencia educativa. Su importancia en el panorama educativo actual destaca por las siguientes razones fundamentales:

1. **Vanguardia e Innovación Educativa** La UNL no solo imparte conocimientos técnicos, sino que está a la vanguardia pedagógica mediante su Proyecto de Innovación Docente (PID). La institución promueve un "Aprendizaje Auténtico" sustituyendo las evaluaciones tradicionales por metodologías activas (Aprendizaje Basado en Problemas, Proyectos e Investigación) y fomenta el uso crítico y responsable de herramientas de Inteligencia Artificial Generativa. Esto garantiza que sus estudiantes desarrollen competencias investigativas, metacognitivas y sociales altamente demandadas en la actualidad.

2. **Inclusión y Diseño Universal para el Aprendizaje (DUA)** Un pilar que le otorga una gran relevancia social a la UNL es su firme compromiso con la inclusión educativa. La universidad implementa el Diseño Universal para el Aprendizaje (DUA) y realiza adaptaciones curriculares no significativas para asegurar la participación equitativa de todos los estudiantes, prestando especial atención a la diversidad cultural y a las necesidades de los grupos de atención prioritaria.

3. **Valores Éticos y Compromiso Social** La formación en la UNL trasciende lo académico. La institución forja profesionales basándose en principios irrenunciables:

   - **Diversidad e Interculturalidad:** Fomentando el respeto y el diálogo de saberes en los distintos contextos culturales.
   - **Igualdad de género y Transparencia:** Actuando con idoneidad, ética y equidad.
   - **Responsabilidad ambiental y sostenibilidad:** Promoviendo la reducción de la huella digital y el cuidado del entorno a través de la tecnología.

4. **Impacto en la Sociedad** La importancia máxima de la Universidad Nacional de Loja se refleja en el perfil de sus egresados, quienes son preparados para interactuar en grupos multidisciplinares y desarrollar proyectos tecnológicos innovadores. El objetivo final de la universidad es que sus profesionales utilicen la ciencia y la tecnología para resolver

problemas reales, optimizar procesos y servir a la sociedad, priorizando siempre a los sectores más vulnerables del Ecuador.

2.1. Introducción a las Estructuras de Datos y Algoritmos

En el desarrollo de software, conocer la sintaxis de un lenguaje de programación es solo el punto de partida. La verdadera potencia de la computación surge de la síntesis: la capacidad de combinar componentes simples y lógica en soluciones elegantes y óptimas. Es aquí donde el estudio de las estructuras de datos y los algoritmos se convierte en el pilar fundamental para cualquier profesional de las ciencias computacionales.

2.1.1. ¿Qué son las Estructuras de Datos?

Una estructura de datos se define como una colección de elementos de datos caracterizados por su organización y por las operaciones definidas para almacenarlos y recuperarlos.

En esencia: Son un conjunto de variables agrupadas para representar un comportamiento específico y facilitar un esquema lógico de manipulación de la información.

A menudo, la mayor dificultad para resolver un problema complejo radica en saber elegir la estructura de datos correcta. Dependiendo de su comportamiento y gestión en la memoria durante la ejecución de un programa, estas se clasifican en:

Tipo	Descripción	Ejemplos
Estructuras Estáticas	Tienen un tamaño de memoria fijo y predefinido antes de la ejecución.	Arreglos (vectores y matrices).
Estructuras Dinámicas	Su tamaño en memoria es variable; puede crecer o reducirse en tiempo de ejecución.	Listas enlazadas, pilas, colas, árboles y grafos.

Además, según el orden de almacenamiento, pueden ser: * **Lineales:** Los datos se almacenan en secuencias adyacentes, uno detrás de otro (ej. arreglos, listas). * **No lineales:** Representan relaciones jerárquicas o de red (ej. árboles y grafos).

2.1.2. Abstracción y Tipos Abstractos de Datos (TAD)

Para manejar la complejidad del mundo real, la informática recurre a la abstracción, que es el proceso de separar las propiedades lógicas de los datos de su implementación física en la máquina.

Esto da lugar al concepto de **Tipo Abstracto de Datos (TAD)**. Un TAD es un modelo matemático o tipo de dato definido por el usuario que especifica un conjunto particular de comportamientos y operaciones (como insertar, buscar o eliminar) de manera completamente independiente de cómo se programen internamente.

- **Ejemplo:** Podemos definir el comportamiento lógico de una "Pila" (bajo la política de que el último en entrar es el primero en salir) y luego decidir si la implementamos físicamente usando un arreglo estático o una lista enlazada dinámica.

2.1.3. Algoritmos y Eficiencia

Las estructuras de datos no existen en el vacío; están íntimamente relacionadas con los algoritmos. Un algoritmo es un conjunto explícito de instrucciones paso a paso diseñado para resolver un problema sin ambigüedades.

Sin embargo, no basta con que un algoritmo funcione. A medida que aumenta la cantidad de datos (escalabilidad), los recursos de tiempo (procesamiento) y espacio (memoria) se vuelven críticos. Para medir y comparar el rendimiento de los algoritmos sin depender del hardware, se utiliza la **Notación Big-O**.

Esta notación permite clasificar la complejidad de un algoritmo a medida que el tamaño de la entrada tiende a infinito:

- $O(1)$: Tiempo constante.
- $O(n)$: Tiempo lineal.
- $O(n^2)$: Tiempo cuadrático.

2.1.4. El Camino hacia la Solución de Problemas

Este libro/curso está diseñado para guiarte en esa transición: dejar de simplemente escribir líneas de código para comenzar a diseñar y sintetizar soluciones.

Al explorar las estructuras de datos desde su diseño lógico (TAD) hasta su implementación física, y al aprender a medir la eficiencia de los algoritmos con notación Big-O, adquirirás las herramientas analíticas necesarias para optimizar procesos y construir el software robusto que la tecnología moderna exige.

3 Capítulo 1: Estructuras Estáticas

3.1. Introducción

```r
# Cargamos las librerías necesarias para las visualizaciones
library(tidyverse)
library(ggplot2)
```

Imagina que tienes una caja de herramientas de tamaño fijo. Sabes exactamente cuántos compartimentos tiene y qué herramienta va en cada uno. No puedes hacer la caja más grande sin comprar una nueva, pero a cambio, encontrar un destornillador específico es instantáneo porque conoces su ubicación exacta. Así es exactamente como funcionan las **estructuras de datos estáticas** en memoria.

En este capítulo, exploraremos dos de las estructuras más fundamentales que comparten esta característica: los **Arreglos** (en memoria principal) y los **Archivos** (en memoria secundaria). Aunque en tus proyectos usarás lenguajes como Java o Python para interactuar con ellos, entender su comportamiento a nivel fundamental te dará el superpoder de escribir código increíblemente rápido y eficiente.

3.2. Arreglos: La Caja de Herramientas Estática

Un arreglo es una colección contigua de elementos del mismo tipo. Su tamaño se define en el momento de la creación y, por naturaleza, no resorte cambios dinámicos.

3.2.1. La Memoria como un Gran Casillero

Pensemos en la RAM como un edificio lleno de casilleros numerados secuencialmente.

```java
// Ilustración en Java de la declaración de un arreglo
int[] calificaciones = new int[5];
```

Cuando declaras ese arreglo en lenguajes fuertemente implementados como Java, el sistema operativo reserva 5 casilleros contiguos. Si la dirección base es la posición lógica `1000`, y cada entero ocupa 4 bytes, el siguiente elemento estará en `1004`, luego `1008`, etc.

Esta contigüidad es la clave de su velocidad extrema de lectura: el tiempo de acceso es $\mathcal{O}(1)$. Veamos una representación abstracta de cómo el tiempo de acceso se mantiene constante sin importar el volumen de la colección:

```
tamanios <- seq(100, 1000, by=100)
tiempo_acceso <- rep(1, length(tamanios)) # Tiempo constante O(1) simulado

ggplot(data.frame(tamanios, tiempo_acceso), aes(x = tamanios, y = tiempo_acceso)) +
  geom_line(color = "steelblue", linewidth = 1.5) +
  theme_minimal() +
  labs(title = "Evolución del Tiempo de Acceso en Arreglos",
       x = "Cantidad de Elementos (N)",
       y = "Tiempo Constante O(1)") +
  ylim(0, 2)
```

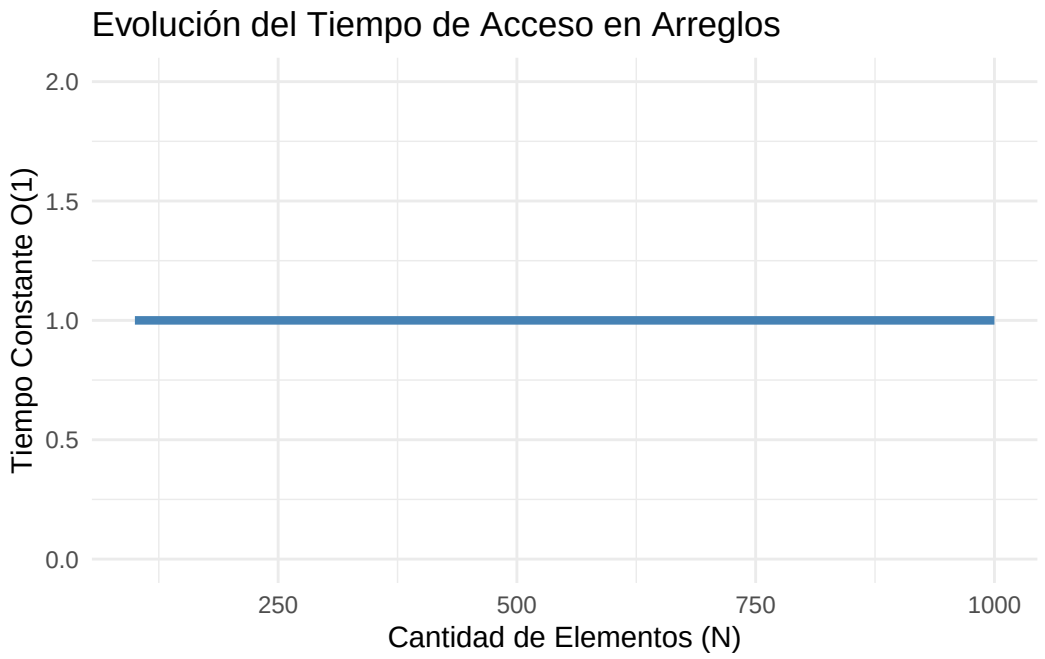

Figura 3.1: Tiempo de acceso constante O(1) en arreglos estáticos al conocer la dirección exacta.

Como observamos en la Figura 3.1, buscar el elemento `500` toma idéntico esfuerzo que buscar el primer elemento. La máquina simplemente ejecuta un cálculo matemático de salto: `Dirección`

`Base + (Índice × Tamaño del Tipo).`

3.2.2. El Lado Oscuro: Inserciones y Eliminaciones

Si bien acceder mediante un índice es magia pura, insertar un nuevo valor en medio del arreglo es un escenario diferente. Es similar a intentar meter un libro ancho en medio de un estante de biblioteca que ya está rebosante de tomos fuertemente empaquetados. Debes desplazar manualmente uno por uno todos los libros de la derecha para abrir un hueco.

Este corrimiento implica un tiempo que escala linealmente u $\mathcal{O}(N)$ en el peor de los casos, preparando el terreno para la imperiosa necesidad de *estructuras dinámicas* que analizaremos más adelante.

3.3. Archivos: Persistencia más allá de la RAM

Mientras que los arreglos viven limitados al horizonte de la memoria volátil, los **Archivos** residen en el almacenamiento secundario. Funcionalmente, actúan como las inmensas bodegas a largo plazo de nuestra aplicación.

Al igual que un arreglo es una tira de variables, un archivo puede interpretarse en su abstracción más simple como un gran arreglo de bytes apilado permanentemente.

```python
# Ejemplo ilusitrativo de cómo en Python el arreglo persistente
# se expone como un stream lineal de datos
with open("datos.txt", "w") as archivo:
    archivo.write("Persistencia consolidada...")
```

Interactuar físicamente con los discos magnéticos o solidos (I/O) siempre conlleva una penalización en latencia masiva comparado con la RAM. Por lo tanto, agrupar las peticiones (usando *buffers* locales basados en arreglos temporales rápidos) es la esencia clave para un I/O de calidad superior.

3.4. Matrices y Arreglos Multidimensionales

Un paso más allá del arreglo unidimensional (vector), encontramos las matrices. Representan datos tabulares (filas y columnas) o espacios N-dimensionales. En memoria, la computadora sigue siendo estrictamente lineal; no existe físicamente una "cuadrícula 2D" dentro de la RAM.

Para lograr esta ilusión de multidimensionalidad, los compiladores y lenguajes de programación utilizan técnicas de transformación matemática, como la representación **Row-Major Order**

(orden contiguo por filas) o la instanciación de "arreglos de arreglos" (el enfoque típico en Java).

3.4.1. Mapeo de 2D a Memoria Lineal

Si tenemos una matriz cuadrática de 3×3, en un enfoque de orden por filas, la primera fila completa ocupará las primeras 3 posiciones contiguas de memoria, seguida inmediatamente por la segunda fila, y así sucesivamente.

La fórmula de traducción de un índice lógico (i, j) a una posición física lineal en una matriz de tamaño $M \times N$ (filas $\times$ columnas), asumiendo índices basados en cero, es:

$Posicin = DireccinBase + (i \times N + j) \times TamaoDelDato$

Esto garantiza que el acceso a cualquier celda de la matriz siga siendo una operación aritmética simple y, por tanto, mantenga un tiempo de acceso $\mathcal{O}(1)$.

```
# Ejemplo en Python usando listas por comprensión para simular
# una matriz (comportamiento 2D) de 3x3 inicializada en 0:
matriz = [[0 for _ in range(3)] for _ in range(3)]
matriz[1][2] = 5 # Acceso lógico bidimensional O(1)
```

3.5. Cadenas de Caracteres (Strings)

Las cadenas de texto, en su formato más fundacional (como en el lenguaje C), no son más que arreglos estáticos de caracteres primitivos (`char`) terminados por un carácter nulo (`\0`).

En lenguajes orientados a objetos modernos como Java y Python, la mayoría de los `Strings` se diseñan como **objetos inmutables**, respaldados internamente bajo el capó por un arreglo estático de caracteres (o bytes para codificaciones como UTF-8).

La Trampa del Rendimiento (Eficiencia $\mathcal{O}(N^2)$)

Esta inmutabilidad tiene una consecuencia directa en el rendimiento algorítmico: cada vez que "modificas" un String (por ejemplo, concatenando un carácter dentro de un bucle `for`), el sistema no modifica el arreglo original porque es estático e inmutable. En su lugar, el sistema se ve forzado a solicitar nueva memoria, crear un **nuevo arreglo estático** más grande, y copiar secuencialmente todos los caracteres viejos más el nuevo.

Si repites esto miles de veces, el rendimiento de tu aplicación se desplomará drásticamente. Esta es la razón técnica por la cual, a nivel de síntesis de software avanzado, se debe utilizar `StringBuilder` en Java o métodos como `''.join(lista)` en Python cuando se construyen concatenaciones masivas de texto.

3.6. Metodologías Activas

3.6.1. Autoevaluación

1. **Reflexiona:** Si necesitas almacenar una lista de usuarios de un *chat-room* activo, donde participantes se desconectan aleatoriamente y entran otros a diferente ritmo; ¿Qué desventajas concretas sufriera tu servidor intentando gestionar esas bajas si solo posees estructuras de tipo arreglo puro?
2. **Experimenta Mentalmente:** Modula la fórmula matemática de un arreglo `DirecciónBase + (Índice * OffsetBytes)`. ¿Qué ocurriría si intentaras buscar `calificaciones[10]` en el arreglo de 5 casillas instanciado antes? ¿Hacia donde apunta la memoria? (Investiga el concepto de *Buffer Overflow* o excepcion *Out of Bounds* de Java).

3.6.2. Resolución de Problemas

El Reto de Fusión Cíclica (Merge Files)

Imagina un caso de la vida real: tienes en posesión dos archivos gigantes, `registros_A.csv` y `registros_B.csv`, y ambos contienen columnas financieras estáticas formadas por IDs crecientes ordenados ascendentemente (1, 2, 5… | 3, 4, 6…). Están ubicados en el lenta memoria secundaria, pero son colosales; no todos los IDs caben dentro del corto cinturón RAM como arreglos masivos.

Tu misión: Escribe pseudocódigo o diseña la arquitectura paso-a-paso de una rutina (estilo de las que implementaríamos en Java o Python) que logre leer ambos archivos *simultáneamente* e ir inyectando sus datos fusionados iterativamente de manera sincronizada en un gran y final `merge.csv` ordenado, utilizando exclusivamente de a 2 casillas temporales en tu RAM mientras iteran.

Este ejercicio fusiona lo mejor de ambos mundos: la inmensa capacidad de iteración de flujos de archivos estáticos y punteros lógicos en arreglos de retención. ¡Presenta tu algoritmo en el siguiente encuentro de codiseño!

4 Estructuras Dinámicas

5 Introducción a las Estructuras Dinámicas en R

¡Hola! En este capítulo, nos sumergiremos en el fascinante mundo de las estructuras de datos dinámicas. A diferencia de las estructuras estáticas como los arrays (o vectores en R) que tienen un tamaño fijo una vez definidos, las estructuras dinámicas pueden crecer o encogerse durante la ejecución del programa, adaptándose a las necesidades cambiantes de los datos.

R, con su enfoque en la manipulación de datos y su poderoso tipo `list`, nos ofrece una gran flexibilidad. Si bien muchas de estas estructuras dinámicas no tienen una implementación "nativa" y explícita como en lenguajes de bajo nivel (como C++ o Java), entender sus principios y cómo simularlas en R es fundamental para escribir código eficiente y estructurado, especialmente cuando se trabaja con datos que cambian de tamaño o cuando se necesita un control preciso sobre el orden de procesamiento.

En este capítulo, exploraremos:

- **Listas Enlazadas**: Desde las simples hasta las dobles y circulares, comprenderemos cómo construir secuencias de elementos que no necesitan estar contiguos en memoria.
- **Pilas (Stacks)**: Una estructura "LIFO" (Last-In, First-Out) que es esencial para tareas como la gestión de funciones o la evaluación de expresiones.
- **Colas (Queues)**: Una estructura "FIFO" (First-In, First-Out) que simula líneas de espera y es crucial en escenarios de procesamiento ordenado.

Prepárate para pensar en cómo R, con su flexibilidad, puede ayudarnos a construir estas abstracciones. Aunque a menudo podemos resolver muchos problemas usando los tipos `vector` o `list` directamente, comprender las estructuras dinámicas nos da una base sólida para elegir la herramienta correcta para cada tarea y apreciar la ingeniería detrás de las soluciones más complejas.

---

6 Listas Enlazadas

Las listas enlazadas son una de las estructuras de datos dinámicas más fundamentales. Consisten en una secuencia de "nodos", donde cada nodo contiene un valor (o datos) y una referencia (o "enlace") al siguiente nodo en la secuencia. La principal ventaja sobre los vectores es que los elementos no necesitan estar almacenados en ubicaciones de memoria contiguas, lo que facilita las operaciones de inserción y eliminación sin necesidad de reasignar o desplazar grandes bloques de memoria.

En R, no tenemos "punteros" explícitos como en C o C++. Sin embargo, podemos simular el comportamiento de las listas enlazadas utilizando la flexibilidad del tipo `list` de R, donde cada "nodo" será una lista que contiene el valor del dato y una referencia al siguiente nodo (que a su vez será otra lista o `NULL`).

6.1. Listas (Generales en R)

Antes de sumergirnos en las listas enlazadas, recordemos cómo funcionan las listas en R, ya que son la base de nuestra simulación. Las listas de R son colecciones ordenadas de objetos, donde los objetos pueden ser de diferentes tipos (heterogéneos).

> **Nota**
>
> Las `list` de R son extremadamente flexibles y, en muchos escenarios, pueden ser suficientes sin necesidad de implementar explícitamente listas enlazadas. Sin embargo, para entender los principios, las implementaremos.

Crear y acceder a elementos de una lista en R es sencillo:

```r
# Crear una lista en R
mi_lista_r <- list(
  nombre = "Alice",
  edad = 30,
  puntuaciones = c(95, 88, 92),
  activa = TRUE
)
```

```r
# Acceder a elementos por nombre
print(mi_lista_r$nombre)
print(mi_lista_r[["edad"]])

# Acceder a elementos por posición
print(mi_lista_r[[3]])

# Modificar un elemento
mi_lista_r$edad <- 31
print(mi_lista_r$edad)

# Añadir un nuevo elemento
mi_lista_r$ciudad <- "Nueva York"
print(mi_lista_r)

# Eliminar un elemento
mi_lista_r$activa <- NULL
print(mi_lista_r)
```

6.2. Listas Simples (Singly Linked Lists)

Una lista simplemente enlazada es la forma más básica de lista enlazada. Cada nodo tiene un valor y un puntero al siguiente nodo. El último nodo apunta a `NULL` (o `NA` en nuestra simulación en R). Solo se puede recorrer hacia adelante.

Estructura de un nodo:

```
[dato | puntero_al_siguiente] -> [dato | puntero_al_siguiente] -> ... -> [dato | NULL]
```

Vamos a simular esto en R:

```r
# Función para crear un nuevo nodo
crear_nodo_simple <- function(valor, siguiente = NULL) {
  list(value = valor, next_node = siguiente)
}

# Clase SLL (Singly Linked List) para encapsular la lógica
SinglyLinkedList <- R6::R6Class(
  "SinglyLinkedList",
  public = list(
```

```r
  head = NULL,

  initialize = function() {
    self$head <- NULL
  },

  # Añadir un nodo al final de la lista
  append = function(valor) {
    new_node <- crear_nodo_simple(valor)
    if (is.null(self$head)) {
      self$head <- new_node
    } else {
      current <- self$head
      while (!is.null(current$next_node)) {
        current <- current$next_node
      }
      current$next_node <- new_node
    }
    invisible(self) # Permite encadenar llamadas
  },

  # Añadir un nodo al principio de la lista
  prepend = function(valor) {
    new_node <- crear_nodo_simple(valor, self$head)
    self$head <- new_node
    invisible(self)
  },

  # Imprimir la lista
  print = function() {
    nodes_str <- c()
    current <- self$head
    while (!is.null(current)) {
      nodes_str <- c(nodes_str, as.character(current$value))
      current <- current$next_node
    }
    cat("Lista Simple: ", paste(nodes_str, collapse = " -> "), "\n")
  },

  # Buscar un valor
  find = function(valor) {
    current <- self$head
```

```r
      while (!is.null(current)) {
        if (identical(current$value, valor)) {
          return(TRUE)
        }
        current <- current$next_node
      }
      return(FALSE)
    },

    # Eliminar un nodo por valor
    delete = function(valor) {
      if (is.null(self$head)) {
        return(invisible(self))
      }

      # Si el nodo a eliminar es la cabeza
      if (identical(self$head$value, valor)) {
        self$head <- self$head$next_node
        return(invisible(self))
      }

      # Buscar el nodo y su predecesor
      current <- self$head
      while (!is.null(current$next_node) && !identical(current$next_node$value, valor)) {
        current <- current$next_node
      }

      # Si se encontró el nodo a eliminar
      if (!is.null(current$next_node)) {
        current$next_node <- current$next_node$next_node
      }
      invisible(self)
    }
  )
)

# Ejemplo de uso de la lista simple
sll <- SinglyLinkedList$new()

sll$append("Manzana")$append("Banana")$append("Cereza")
sll$print()
```

```r
sll$prepend("Uva")
sll$print()

cat("¿Banana está en la lista?", sll$find("Banana"), "\n")
cat("¿Naranja está en la lista?", sll$find("Naranja"), "\n")

sll$delete("Banana")
sll$print()

sll$delete("Uva") # Eliminar la cabeza
sll$print()

sll$delete("Cereza") # Eliminar el último
sll$print()

sll$delete("Manzana") # Eliminar el último que queda
sll$print()
```

> **Nota**
>
> **Nota sobre la simulación en R:** Al crear un nodo como `list(value = valor, next_node = siguiente)`, el `siguiente` es una referencia *copia* del objeto `list` completo. R gestiona la memoria y las referencias de objetos de manera diferente a lenguajes como C, donde un puntero es una dirección de memoria. Sin embargo, para entender el comportamiento lógico de las listas enlazadas, esta simulación es muy efectiva. Para grandes volúmenes de datos y operaciones frecuentes, las `list` nativas de R o estructuras más optimizadas podrían ser preferibles por rendimiento.

6.3. Listas Dobles (Doubly Linked Lists)

Una lista doblemente enlazada mejora la lista simple añadiendo un puntero al nodo *anterior* además del puntero al siguiente nodo. Esto permite recorrer la lista en ambas direcciones y simplifica ciertas operaciones, como la eliminación de un nodo (ya que no necesitamos mantener un puntero al predecesor).

Estructura de un nodo:

```
NULL <- [puntero_anterior | dato | puntero_siguiente] <-> ... <-> [puntero_anterior | dato |
```

```r
# Función para crear un nuevo nodo doble
crear_nodo_doble <- function(valor, anterior = NULL, siguiente = NULL) {
  list(value = valor, prev_node = anterior, next_node = siguiente)
}

# Clase DLL (Doubly Linked List)
DoublyLinkedList <- R6::R6Class(
  "DoublyLinkedList",
  public = list(
    head = NULL,
    tail = NULL, # Mantenemos un puntero a la cola para inserciones rápidas al final

    initialize = function() {
      self$head <- NULL
      self$tail <- NULL
    },

    # Añadir un nodo al final
    append = function(valor) {
      new_node <- crear_nodo_doble(valor)
      if (is.null(self$head)) {
        self$head <- new_node
        self$tail <- new_node
      } else {
        self$tail$next_node <- new_node
        new_node$prev_node <- self$tail
        self$tail <- new_node
      }
      invisible(self)
    },

    # Añadir un nodo al principio
    prepend = function(valor) {
      new_node <- crear_nodo_doble(valor, siguiente = self$head)
      if (is.null(self$head)) {
        self$head <- new_node
        self$tail <- new_node
      } else {
        self$head$prev_node <- new_node
        self$head <- new_node
      }
      invisible(self)
```

```r
  },

  # Imprimir la lista (hacia adelante)
  print_forward = function() {
    nodes_str <- c()
    current <- self$head
    while (!is.null(current)) {
      nodes_str <- c(nodes_str, as.character(current$value))
      current <- current$next_node
    }
    cat("Lista Doble (Adelante): ", paste(nodes_str, collapse = " <-> "), "\n")
  },

  # Imprimir la lista (hacia atrás)
  print_backward = function() {
    nodes_str <- c()
    current <- self$tail
    while (!is.null(current)) {
      nodes_str <- c(nodes_str, as.character(current$value))
      current <- current$prev_node
    }
    cat("Lista Doble (Atrás): ", paste(nodes_str, collapse = " <-> "), "\n")
  },

  # Eliminar un nodo por valor
  delete = function(valor) {
    if (is.null(self$head)) {
      return(invisible(self))
    }

    current <- self$head
    while (!is.null(current) && !identical(current$value, valor)) {
      current <- current$next_node
    }

    # Si el valor no se encontró
    if (is.null(current)) {
      return(invisible(self))
    }

    # Si el nodo a eliminar es la cabeza
    if (is.null(current$prev_node)) {
```

```
        self$head <- current$next_node
        if (!is.null(self$head)) {
          self$head$prev_node <- NULL
        } else { # Si la lista queda vacía
          self$tail <- NULL
        }
      }
      # Si el nodo a eliminar es la cola
      else if (is.null(current$next_node)) {
        self$tail <- current$prev_node
        if (!is.null(self$tail)) {
          self$tail$next_node <- NULL
        } else { # Si la lista queda vacía
          self$head <- NULL
        }
      }
      # Si el nodo está en el medio
      else {
        current$prev_node$next_node <- current$next_node
        current$next_node$prev_node <- current$prev_node
      }
      invisible(self)
    }
  )
)

# Ejemplo de uso de la lista doble
dll <- DoublyLinkedList$new()

dll$append("Uno")$append("Dos")$append("Tres")
dll$print_forward()
dll$print_backward()

dll$prepend("Cero")
dll$print_forward()

dll$delete("Dos")
dll$print_forward()
dll$print_backward()

dll$delete("Cero") # Eliminar la cabeza
dll$print_forward()
```

```r
dll$delete("Tres") # Eliminar la cola
dll$print_forward()

dll$delete("Uno") # Eliminar el último
dll$print_forward()
dll$print_backward() # La lista está vacía
```

6.4. Listas Circulares (Circular Linked Lists)

Una lista circular es una lista enlazada donde el último nodo apunta al primer nodo, creando un ciclo. Puede ser simplemente o doblemente enlazada, pero la característica clave es el cierre del ciclo. Esto es útil para estructuras que no tienen un "principio" o "fin" natural, o donde se necesita un acceso continuo a los elementos (por ejemplo, en la planificación de procesos de un sistema operativo).

Estructura (Simple Circular):

```
[dato | puntero_siguiente] -> [dato | puntero_siguiente] -> ... -> [dato | puntero_al_primero
   ^                                                                |
   |________________________________________________________________|
```

```r
# Clase CLL (Circular Linked List)
CircularLinkedList <- R6::R6Class(
  "CircularLinkedList",
  public = list(
    head = NULL,

    initialize = function() {
      self$head <- NULL
    },

    # Añadir un nodo al final (mantiene el orden circular)
    append = function(valor) {
      new_node <- crear_nodo_simple(valor) # Usamos el nodo simple para la base
      if (is.null(self$head)) {
        self$head <- new_node
        new_node$next_node <- self$head # Apunta a sí mismo para formar el círculo
      } else {
        current <- self$head
        # Recorrer hasta el nodo justo antes de la cabeza (el actual "final")
```

```
      while (!identical(current$next_node, self$head)) {
        current <- current$next_node
      }
      current$next_node <- new_node
      new_node$next_node <- self$head # El nuevo nodo apunta a la cabeza
    }
    invisible(self)
  },

  # Imprimir la lista (recorre una vez el círculo)
  print = function() {
    if (is.null(self$head)) {
      cat("Lista Circular: Vacía\n")
      return(invisible(self))
    }

    nodes_str <- c()
    current <- self$head
    repeat {
      nodes_str <- c(nodes_str, as.character(current$value))
      current <- current$next_node
      if (identical(current, self$head)) { # Detener cuando volvemos a la cabeza
        break
      }
    }
    cat("Lista Circular: ", paste(nodes_str, collapse = " -> "), " -> (vuelve a ", nodes_st
  },

  # Eliminar un nodo por valor
  delete = function(valor) {
    if (is.null(self$head)) {
      return(invisible(self))
    }

    # Si la lista tiene un solo nodo
    if (identical(self$head$next_node, self$head) && identical(self$head$value, valor)) {
      self$head <- NULL
      return(invisible(self))
    }

    current <- self$head
    prev <- NULL
```

```r
      # Buscar el nodo a eliminar
      repeat {
        if (identical(current$value, valor)) {
          break # Se encontró el nodo
        }
        prev <- current
        current <- current$next_node
        if (identical(current, self$head)) { # Se recorrió toda la lista y no se encontró
          return(invisible(self))
        }
      }

      # Si el nodo a eliminar es la cabeza
      if (identical(current, self$head)) {
        # Encontrar el último nodo para actualizar su puntero
        temp_tail <- self$head
        while (!identical(temp_tail$next_node, self$head)) {
          temp_tail <- temp_tail$next_node
        }
        self$head <- self$head$next_node
        temp_tail$next_node <- self$head
      } else { # El nodo a eliminar no es la cabeza
        prev$next_node <- current$next_node
      }
      invisible(self)
    }
  )
)

# Ejemplo de uso de la lista circular
cll <- CircularLinkedList$new()

cll$append("Alfa")$append("Beta")$append("Gamma")
cll$print()

cll$delete("Beta")
cll$print()

cll$delete("Alfa") # Eliminar la cabeza
cll$print()

cll$delete("Gamma") # Eliminar el último que queda
```

```
cll$print()

cll$append("Único")
cll$print()
cll$delete("Único")
cll$print() # La lista está vacía
```

---

7 Pilas (Stacks)

Una Pila es una estructura de datos lineal que sigue el principio "LIFO" (Last-In, First-Out), es decir, el último elemento añadido es el primero en ser eliminado. Piensa en una pila de platos: solo puedes añadir un plato encima y solo puedes quitar el plato de más arriba.

Las operaciones principales de una pila son:

- **Push**: Añadir un elemento a la parte superior de la pila.
- **Pop**: Eliminar y devolver el elemento de la parte superior de la pila.
- **Peek/Top**: Devolver el elemento de la parte superior sin eliminarlo.
- **isEmpty**: Comprobar si la pila está vacía.
- **Size**: Devolver el número de elementos en la pila.

En R, podemos implementar una pila de manera muy eficiente usando un `vector` o una `list` estándar, aprovechando que las operaciones de añadir/eliminar al final de un vector/lista son muy rápidas.

```
# Clase Stack
Stack <- R6::R6Class(
  "Stack",
  public = list(
    elements = NULL,

    initialize = function() {
      self$elements <- list() # Usamos una lista para mayor flexibilidad de tipos
    },

    # Añadir un elemento a la pila (Push)
    push = function(item) {
      self$elements <- c(self$elements, list(item)) # Añadir al final
      invisible(self)
    },

    # Eliminar y devolver el elemento superior (Pop)
    pop = function() {
      if (self$is_empty()) {
        stop("La pila está vacía, no se puede hacer pop.")
```

```
      }
      last_index <- length(self$elements)
      item <- self$elements[[last_index]]
      self$elements <- self$elements[-last_index] # Eliminar el último
      return(item)
    },

    # Devolver el elemento superior sin eliminarlo (Peek/Top)
    peek = function() {
      if (self$is_empty()) {
        return(NULL) # O podrías lanzar un error, dependiendo del diseño
      }
      return(self$elements[[length(self$elements)]])
    },

    # Comprobar si la pila está vacía
    is_empty = function() {
      return(length(self$elements) == 0)
    },

    # Devolver el tamaño de la pila
    size = function() {
      return(length(self$elements))
    },

    # Imprimir el contenido de la pila
    print = function() {
      if (self$is_empty()) {
        cat("Pila: Vacía\n")
      } else {
        cat("Pila (Top -> Base): ", paste(rev(unlist(self$elements)), collapse = " | "), "\n"
      }
    }
  )
)

# Ejemplo de uso de la pila
mi_pila <- Stack$new()

mi_pila$push("Primero")
mi_pila$push("Segundo")
mi_pila$push("Tercero")
```

```r
mi_pila$print()

cat("Elemento superior (peek):", mi_pila$peek(), "\n")
cat("Tamaño de la pila:", mi_pila$size(), "\n")

item_sacado <- mi_pila$pop()
cat("Sacado de la pila (pop):", item_sacado, "\n")
mi_pila$print()

mi_pila$push("Cuarto")
mi_pila$print()

while (!mi_pila$is_empty()) {
  cat("Sacando:", mi_pila$pop(), "\n")
}
mi_pila$print()

# Intentar pop en pila vacía (esto causará un error)
# tryCatch({
#   mi_pila$pop()
# }, error = function(e) {
#   message("Error al hacer pop en pila vacía: ", e$message)
# })
```

---

8 Colas (Queues)

Una Cola es una estructura de datos lineal que sigue el principio "FIFO" (First-In, First-Out), es decir, el primer elemento añadido es el primero en ser eliminado. Piensa en una cola de personas en un supermercado: la primera persona en llegar es la primera en ser atendida.

Las operaciones principales de una cola son:

- **Enqueue**: Añadir un elemento al final de la cola.
- **Dequeue**: Eliminar y devolver el elemento del frente de la cola.
- **Front**: Devolver el elemento del frente sin eliminarlo.
- **isEmpty**: Comprobar si la cola está vacía.
- **Size**: Devolver el número de elementos en la cola.

La implementación de una cola en R utilizando `vector` o `list` requiere un poco más de consideración que la pila. Añadir al final (`enqueue`) es eficiente. Sin embargo, eliminar del principio (`dequeue`) en un vector grande puede ser menos eficiente, ya que R tiene que "desplazar" todos los elementos restantes para llenar el hueco. Para colas de gran tamaño y operaciones frecuentes de `dequeue`, una implementación basada en una lista enlazada (o una estructura optimizada como `collections::deque` si está disponible) sería más performante. Para propósitos educativos y colas de tamaño moderado, la lista de R es suficiente.

```r
# Clase Queue
Queue <- R6::R6Class(
  "Queue",
  public = list(
    elements = NULL,

    initialize = function() {
      self$elements <- list() # Usamos una lista
    },

    # Añadir un elemento a la cola (Enqueue)
    enqueue = function(item) {
      self$elements <- c(self$elements, list(item)) # Añadir al final
      invisible(self)
    },
```

```
    # Eliminar y devolver el elemento del frente (Dequeue)
    dequeue = function() {
      if (self$is_empty()) {
        stop("La cola está vacía, no se puede hacer dequeue.")
      }
      item <- self$elements[[1]] # Obtener el primer elemento
      self$elements <- self$elements[-1] # Eliminar el primer elemento (¡cuidado con rendimie
      return(item)
    },

    # Devolver el elemento del frente sin eliminarlo (Front)
    front = function() {
      if (self$is_empty()) {
        return(NULL)
      }
      return(self$elements[[1]])
    },

    # Comprobar si la cola está vacía
    is_empty = function() {
      return(length(self$elements) == 0)
    },

    # Devolver el tamaño de la cola
    size = function() {
      return(length(self$elements))
    },

    # Imprimir el contenido de la cola
    print = function() {
      if (self$is_empty()) {
        cat("Cola: Vacía\n")
      } else {
        cat("Cola (Frente -> Final): ", paste(unlist(self$elements), collapse = " | "), "\n")
      }
    }
  )
)

# Ejemplo de uso de la cola
mi_cola <- Queue$new()
```

```r
mi_cola$enqueue("Cliente 1")
mi_cola$enqueue("Cliente 2")
mi_cola$enqueue("Cliente 3")
mi_cola$print()

cat("Elemento al frente (front):", mi_cola$front(), "\n")
cat("Tamaño de la cola:", mi_cola$size(), "\n")

cliente_atendido <- mi_cola$dequeue()
cat("Atendiendo a (dequeue):", cliente_atendido, "\n")
mi_cola$print()

mi_cola$enqueue("Cliente 4")
mi_cola$print()

while (!mi_cola$is_empty()) {
  cat("Atendiendo a:", mi_cola$dequeue(), "\n")
}
mi_cola$print()

# Intentar dequeue en cola vacía (esto causará un error)
# tryCatch({
#   mi_cola$dequeue()
# }, error = function(e) {
#   message("Error al hacer dequeue en cola vacía: ", e$message)
# })
```

---

9 Conclusión

¡Felicidades! Has recorrido un camino fundamental en el estudio de las estructuras de datos dinámicas. Hemos explorado las listas enlazadas en sus variantes (simples, dobles, circulares), comprendiendo cómo, aunque R no tenga "punteros" explícitos, podemos simular su comportamiento para entender sus principios y ventajas en la gestión de memoria y flexibilidad.

También hemos implementado las Pilas (LIFO) y las Colas (FIFO), dos estructuras ubicuas en la informática, mostrando cómo R, con sus tipos `list` y `vector`, puede manejarlas de manera eficiente para muchos casos de uso. Hemos destacado las implicaciones de rendimiento, especialmente al realizar `dequeue` en una cola basada en vectores R estándar.

Entender estas estructuras no solo te proporciona herramientas para resolver problemas específicos, sino que también mejora tu comprensión de cómo funcionan los datos subyacentes en muchos algoritmos y sistemas. Aunque R a menudo abstrae estas complejidades con sus potentes funciones y tipos de datos de alto nivel, el conocimiento profundo de estas bases te convertirá en un programador de R más versátil y perspicaz. ¡Sigue explorando y construyendo!

10 Algoritmos de Ordenación

11 Algoritmos de Ordenación

¡Bienvenidos a este capítulo fundamental en la ciencia de la computación y la programación con R! La ordenación de datos es una de las operaciones más comunes y críticas que realizamos. Desde organizar una tabla de datos por una columna específica hasta optimizar algoritmos más complejos, la capacidad de ordenar eficientemente es indispensable.

Comprender cómo funcionan los diferentes algoritmos de ordenación no solo mejora nuestra capacidad para escribir código eficiente, sino que también profundiza nuestra comprensión de la complejidad algorítmica, la eficiencia de los recursos y la lógica de programación.

En este capítulo, exploraremos algunos de los algoritmos de ordenación más conocidos y fundamentales: Burbuja (Bubble Sort), Inserción (Insertion Sort), Mergesort y Quicksort. Analizaremos su funcionamiento, implementaremos cada uno en R con bloques de código ejecutables y discutiremos sus características de rendimiento, incluyendo su complejidad temporal y espacial.

11.1. Burbuja (Bubble Sort)

11.1.1. Concepto

El algoritmo de ordenación por burbuja es uno de los más sencillos de entender e implementar. Su nombre proviene del hecho de que los elementos más "grandes" (o más "pesados", si imaginamos burbujas) "flotan" gradualmente hacia el final de la lista a medida que se realizan comparaciones y posibles intercambios.

El algoritmo funciona recorriendo repetidamente la lista, comparando cada par de elementos adyacentes y cambiándolos de posición si están en el orden incorrecto. Este proceso se repite hasta que ya no se necesiten intercambios en una pasada completa, lo que indica que la lista está ordenada.

11.1.2. Implementación en R

A continuación, implementaremos `bubble_sort` en R.

```r
bubble_sort <- function(arr) {
  n <- length(arr)

  # Bucle exterior para cada pasada (n-1 pasadas como máximo)
  for (i in 1:(n - 1)) {
    swapped <- FALSE # Bandera para optimización: si no hay intercambios, está ordenado

    # Bucle interior para comparaciones y swaps
    # En cada pasada, el elemento más grande "burbujea" a su posición final
    # Por lo tanto, no necesitamos comparar los últimos i elementos
    for (j in 1:(n - i)) {
      if (arr[j] > arr[j + 1]) {
        # Intercambiar elementos
        temp <- arr[j]
        arr[j] <- arr[j + 1]
        arr[j + 1] <- temp
        swapped <- TRUE
      }
    }

    # Si no hubo intercambios en esta pasada, la lista está ordenada
    if (!swapped) {
      break
    }
  }
  return(arr)
}
```

11.1.3. Explicación del Código

1. **`bubble_sort(arr)`**: Nuestra función toma un vector **`arr`** como entrada.
2. **`n <- length(arr)`**: Obtenemos la longitud del vector.
3. **`for (i in 1:(n - 1))`**: Este bucle exterior controla el número de pasadas a través de la lista. En el peor de los casos, necesitaremos **`n-1`** pasadas para asegurar que todos los elementos estén en su lugar.
4. **`swapped <- FALSE`**: Esta bandera es una optimización. Si una pasada completa no realiza ningún intercambio, significa que la lista ya está ordenada, y podemos salir del bucle exterior prematuramente.
5. **`for (j in 1:(n - i))`**: Este bucle interior es el corazón del algoritmo. Recorre los elementos adyacentes.

- La condición `n - i` es crucial: después de la primera pasada (`i=1`), el elemento más grande ya está en la última posición. Después de la segunda pasada (`i=2`), los dos elementos más grandes están en las dos últimas posiciones, y así sucesivamente. Por lo tanto, no necesitamos comparar los últimos `i` elementos que ya están ordenados.

6. **`if (arr[j] > arr[j + 1])`**: Compara si el elemento actual es mayor que el siguiente. Si lo es, están en el orden incorrecto.
7. **Intercambio**: Si la condición es verdadera, se realiza un intercambio simple utilizando una variable temporal `temp`.
8. **`if (!swapped) break`**: Si después de una pasada completa del bucle interior, `swapped` sigue siendo `FALSE`, significa que no se realizaron intercambios, y la lista ya está ordenada. Rompemos el bucle exterior para mejorar la eficiencia.
9. **`return(arr)`**: La función devuelve el vector ordenado.

11.1.4. Complejidad

- **Tiempo**:
  - **Peor caso**: O(n^2). Ocurre cuando el array está ordenado en orden inverso. Cada elemento necesita "burbujear" a través de casi toda la lista.
  - **Caso promedio**: O(n^2).
  - **Mejor caso**: O(n). Si el array ya está ordenado, la optimización con la bandera `swapped` hará que el algoritmo complete en una sola pasada.
- **Espacio**: O(1). Es un algoritmo de ordenación "in-place" ya que solo utiliza una cantidad constante de memoria adicional para la variable `temp`.

11.1.5. Ejemplo

Veamos `bubble_sort` en acción con algunos datos de ejemplo.

```r
# Ejemplo 1: Un vector desordenado
vector_desordenado <- c(5, 1, 4, 2, 8)
print(paste("Vector original:", paste(vector_desordenado, collapse = ", ")))
```

```
[1] "Vector original: 5, 1, 4, 2, 8"
```

```r
vector_ordenado <- bubble_sort(vector_desordenado)
print(paste("Vector ordenado (Bubble Sort):", paste(vector_ordenado, collapse = ", ")))
```

```
[1] "Vector ordenado (Bubble Sort): 1, 2, 4, 5, 8"
```

```r
# Ejemplo 2: Un vector ya ordenado (para ver la optimización)
vector_ordenado_previo <- c(1, 2, 3, 4, 5)
print(paste("Vector original (ya ordenado):", paste(vector_ordenado_previo, collapse = ", ")
```

```
[1] "Vector original (ya ordenado): 1, 2, 3, 4, 5"
```

```r
vector_ordenado_opt <- bubble_sort(vector_ordenado_previo)
print(paste("Vector ordenado (Bubble Sort):", paste(vector_ordenado_opt, collapse = ", ")))
```

```
[1] "Vector ordenado (Bubble Sort): 1, 2, 3, 4, 5"
```

```r
# Ejemplo 3: Un vector con elementos duplicados
vector_duplicados <- c(7, 2, 5, 2, 1, 9)
print(paste("Vector con duplicados:", paste(vector_duplicados, collapse = ", ")))
```

```
[1] "Vector con duplicados: 7, 2, 5, 2, 1, 9"
```

```r
vector_ordenado_dup <- bubble_sort(vector_duplicados)
print(paste("Vector ordenado (Bubble Sort):", paste(vector_ordenado_dup, collapse = ", ")))
```

```
[1] "Vector ordenado (Bubble Sort): 1, 2, 2, 5, 7, 9"
```

11.2. Inserción (Insertion Sort)

11.2.1. Concepto

El algoritmo de ordenación por inserción funciona de manera similar a cómo ordenaríamos un mazo de cartas en nuestra mano. Recorremos la lista elemento por elemento, tomando cada elemento y "insertándolo" en la posición correcta dentro de la parte ya ordenada de la lista.

La lista se divide conceptualmente en dos partes: una sublista ordenada (inicialmente solo el primer elemento) y el resto de elementos desordenados. En cada paso, se toma el primer elemento de la parte desordenada y se compara con los elementos de la sublista ordenada, desplazando los elementos mayores que él para hacer espacio y luego insertándolo en su posición correcta.

11.2.2. Implementación en R

Implementaremos `insertion_sort` en R.

```
insertion_sort <- function(arr) {
  n <- length(arr)

  # Itera desde el segundo elemento hasta el final
  for (i in 2:n) {
    key <- arr[i] # Elemento actual a insertar
    j <- i - 1     # Índice del último elemento de la sublista ordenada

    # Mueve los elementos de arr[0..i-1], que son mayores que key,
    # una posición adelante de su posición actual
    while (j >= 1 && arr[j] > key) {
      arr[j + 1] <- arr[j]
      j <- j - 1
    }

    # Inserta key en su posición correcta
    arr[j + 1] <- key
  }
  return(arr)
}
```

11.2.3. Explicación del Código

1. **`insertion_sort(arr)`**: Nuestra función toma un vector `arr` como entrada.
2. **`n <- length(arr)`**: Obtenemos la longitud del vector.
3. **`for (i in 2:n)`**: Este bucle exterior comienza desde el segundo elemento (`i=2`) porque asumimos que el primer elemento (`arr[1]`) ya es una sublista ordenada de un solo elemento.
4. **`key <- arr[i]`**: `key` es el elemento actual que queremos insertar en la sublista ordenada.
5. **`j <- i - 1`**: `j` es el índice del último elemento de la sublista ya ordenada.
6. **`while (j >= 1 && arr[j] > key)`**: Este bucle `while` se encarga de:
   - Comparar `key` con los elementos de la sublista ordenada (`arr[j]`).
   - Si un elemento `arr[j]` es mayor que `key`, lo desplaza una posición a la derecha (`arr[j + 1] <- arr[j]`).
   - `j` se decrementa para moverse hacia el inicio de la sublista ordenada, buscando el lugar correcto para `key`.

7. **`arr[j + 1] <- key`**: Una vez que el bucle `while` termina (ya sea porque `j` llegó a 0 o porque `arr[j]` es menor o igual que `key`), significa que `j + 1` es la posición correcta para insertar `key`.
8. **`return(arr)`**: La función devuelve el vector ordenado.

11.2.4. Complejidad

- **Tiempo**:
  - **Peor caso**: O(n^2). Ocurre cuando el array está ordenado en orden inverso. Cada elemento debe ser comparado y desplazado a través de casi toda la sublista ordenada.
  - **Caso promedio**: O(n^2).
  - **Mejor caso**: O(n). Si el array ya está ordenado, cada elemento se compara solo con el elemento adyacente anterior, lo que resulta en un tiempo lineal.
- **Espacio**: O(1). Es un algoritmo "in-place".

11.2.5. Ejemplo

Veamos `insertion_sort` en acción.

```r
# Ejemplo 1: Un vector desordenado
vector_desordenado <- c(12, 11, 13, 5, 6)
print(paste("Vector original:", paste(vector_desordenado, collapse = ", ")))
```

```
[1] "Vector original: 12, 11, 13, 5, 6"
```

```r
vector_ordenado <- insertion_sort(vector_desordenado)
print(paste("Vector ordenado (Insertion Sort):", paste(vector_ordenado, collapse = ", ")))
```

```
[1] "Vector ordenado (Insertion Sort): 5, 6, 11, 12, 13"
```

```r
# Ejemplo 2: Un vector pequeño
vector_pequeño <- c(4, 1, 3, 2)
print(paste("Vector pequeño:", paste(vector_pequeño, collapse = ", ")))
```

```
[1] "Vector pequeño: 4, 1, 3, 2"
```

```r
vector_ordenado_p <- insertion_sort(vector_pequeño)
print(paste("Vector ordenado (Insertion Sort):", paste(vector_ordenado_p, collapse = ", ")))
```

```
[1] "Vector ordenado (Insertion Sort): 1, 2, 3, 4"
```

11.3. Mergesort

11.3.1. Concepto

Mergesort es un algoritmo de ordenación eficiente y estable que sigue el paradigma "divide y vencerás". Su principio es dividir el array de entrada en dos mitades, ordenar cada mitad recursivamente, y luego combinar (fusionar) las dos mitades ordenadas para producir el array final ordenado.

Los pasos principales son: 1. **Dividir**: Si la lista tiene más de un elemento, divídela en dos sublistas aproximadamente iguales. 2. **Conquistar**: Ordena recursivamente cada sublista utilizando Mergesort. 3. **Combinar (Merge)**: Fusiona las dos sublistas ordenadas en una única lista ordenada. La fase de combinación es donde ocurre la mayor parte del trabajo.

La recursión termina cuando la sublista tiene un solo elemento, ya que una lista de un solo elemento se considera ordenada por definición.

11.3.2. Implementación en R

Necesitaremos dos funciones: una para la lógica recursiva de división (`merge_sort`) y otra para la combinación de dos arrays ya ordenados (`merge_arrays`).

```r
# Función para combinar dos sub-arrays ordenados
merge_arrays <- function(left, right) {
  result <- c()

  # Mientras ambos arrays tengan elementos
  while (length(left) > 0 && length(right) > 0) {
    if (left[1] <= right[1]) {
      result <- c(result, left[1])
      left <- left[-1] # Elimina el primer elemento
    } else {
      result <- c(result, right[1])
      right <- right[-1] # Elimina el primer elemento
    }
```

```
  }

  # Añade los elementos restantes, si los hay
  if (length(left) > 0) {
    result <- c(result, left)
  }
  if (length(right) > 0) {
    result <- c(result, right)
  }

  return(result)
}

# Función principal de Mergesort
merge_sort <- function(arr) {
  n <- length(arr)

  # Caso base: un array con 0 o 1 elemento ya está ordenado
  if (n <= 1) {
    return(arr)
  }

  # Divide el array en dos mitades
  mid <- floor(n / 2)
  left_half <- arr[1:mid]
  right_half <- arr[(mid + 1):n]

  # Ordena recursivamente cada mitad
  sorted_left <- merge_sort(left_half)
  sorted_right <- merge_sort(right_half)

  # Combina las mitades ordenadas
  return(merge_arrays(sorted_left, sorted_right))
}
```

11.3.3. Explicación del Código

11.3.3.1. `merge_arrays(left, right)`:

1. **`result <- c()`**: Inicializamos un vector vacío para almacenar el resultado combinado.

2. `while (length(left) > 0 && length(right) > 0)`: Este bucle compara los primeros elementos de `left` y `right`.
   - El elemento más pequeño se añade a `result` y se elimina de su array original usando `left[-1]` o `right[-1]`. **Nota**: La eliminación de elementos al inicio de un vector en R es una operación costosa O(n) que puede impactar el rendimiento real de esta implementación en R. En lenguajes de bajo nivel se usarían punteros o índices.
3. `if (length(left) > 0)` / `if (length(right) > 0)`: Después del bucle `while`, uno de los arrays podría tener elementos restantes (porque uno se agotó antes que el otro). Estos elementos restantes ya están ordenados y simplemente se añaden al final de `result`.
4. `return(result)`: Devuelve el array combinado y ordenado.

11.3.3.2. `merge_sort(arr)`:

1. `n <- length(arr)`: Obtiene la longitud del array.
2. `if (n <= 1)`: Este es el **caso base** de la recursión. Si el array tiene 0 o 1 elemento, ya está ordenado, así que lo devolvemos tal cual.
3. `mid <- floor(n / 2)`: Calcula el punto medio para dividir el array.
4. `left_half <- arr[1:mid]` y `right_half <- arr[(mid + 1):n]`: Crea las dos submitades del array.
5. `sorted_left <- merge_sort(left_half)` y `sorted_right <- merge_sort(right_half)`: Llama recursivamente a `merge_sort` en cada mitad hasta que se alcanzan los casos base (arrays de un solo elemento).
6. `return(merge_arrays(sorted_left, sorted_right))`: Una vez que las mitades se han ordenado recursivamente, se fusionan utilizando la función `merge_arrays`.

11.3.4. Complejidad

- **Tiempo**:
  - **Peor caso**: O(n log n).
  - **Caso promedio**: O(n log n).
  - **Mejor caso**: O(n log n). Mergesort tiene un rendimiento consistente en todos los casos debido a su estrategia de división equitativa y la naturaleza de la operación de fusión. El factor `log n` viene de las divisiones recursivas, y `n` de la operación de fusión en cada nivel.
- **Espacio**: O(n). Mergesort no es un algoritmo "in-place" puro. Requiere espacio adicional para almacenar las sublistas temporales durante la fase de fusión. En nuestra implementación de R, la creación de nuevos vectores en cada paso de `merge_arrays` contribuye a esta complejidad espacial.

11.3.5. Ejemplo

Veamos `merge_sort` en acción.

```
# Ejemplo 1: Un vector desordenado
vector_desordenado <- c(38, 27, 43, 3, 9, 82, 10)
print(paste("Vector original:", paste(vector_desordenado, collapse = ", ")))
```

```
[1] "Vector original: 38, 27, 43, 3, 9, 82, 10"
```

```
vector_ordenado <- merge_sort(vector_desordenado)
print(paste("Vector ordenado (Mergesort):", paste(vector_ordenado, collapse = ", ")))
```

```
[1] "Vector ordenado (Mergesort): 3, 9, 10, 27, 38, 43, 82"
```

```
# Ejemplo 2: Un vector con elementos duplicados
vector_duplicados <- c(5, 2, 8, 2, 1, 5, 9)
print(paste("Vector con duplicados:", paste(vector_duplicados, collapse = ", ")))
```

```
[1] "Vector con duplicados: 5, 2, 8, 2, 1, 5, 9"
```

```
vector_ordenado_dup <- merge_sort(vector_duplicados)
print(paste("Vector ordenado (Mergesort):", paste(vector_ordenado_dup, collapse = ", ")))
```

```
[1] "Vector ordenado (Mergesort): 1, 2, 2, 5, 5, 8, 9"
```

11.4. Quicksort

11.4.1. Concepto

Quicksort es otro algoritmo de ordenación eficiente basado en el paradigma "divide y vencerás". Es considerado uno de los algoritmos de ordenación más rápidos en la práctica para grandes conjuntos de datos no estructurados.

El funcionamiento básico es el siguiente: 1. **Elegir un Pivote**: Selecciona un elemento del array, llamado "pivote". La elección del pivote es crucial para el rendimiento. 2. **Particionar**: Reordena el array de modo que todos los elementos con valores menores que el pivote queden a su izquierda, y todos los elementos con valores mayores queden a su derecha. Los elementos iguales al pivote pueden ir a cualquiera de los dos lados. Después de la partición, el pivote está

en su posición final ordenada. 3. **Recursión**: Aplica Quicksort recursivamente a la sublista de elementos con valores menores que el pivote y a la sublista de elementos con valores mayores que el pivote.

La recursión termina cuando una sublista tiene cero o un elemento, ya que ya están ordenados.

11.4.2. Implementación en R

Implementaremos `quick_sort` en R. Esta implementación es una versión simplificada que crea nuevos vectores en cada paso de la partición, lo cual es más idiomático en R pero menos eficiente en espacio que una implementación "in-place" con punteros.

```
quick_sort <- function(arr) {
  n <- length(arr)

  # Caso base: un array con 0 o 1 elemento ya está ordenado
  if (n <= 1) {
    return(arr)
  }

  # Elegir un pivote. Aquí usamos el último elemento por simplicidad.
  # Otras estrategias incluyen elegir el primero, el del medio, o un pivote aleatorio.
  pivot_index <- n
  pivot <- arr[pivot_index]

  # Separar los elementos menores y mayores que el pivote
  # Excluimos el pivote del array original antes de la partición
  remaining_elements <- arr[-pivot_index]

  # Utilizar un enfoque R-idiomático para la partición
  # Esto crea nuevos vectores, lo que impacta la eficiencia espacial
  less <- remaining_elements[remaining_elements < pivot]
  greater <- remaining_elements[remaining_elements >= pivot] # >= para incluir iguales o mayo

  # Ordenar recursivamente las sublistas y combinar
  return(c(quick_sort(less), pivot, quick_sort(greater)))
}
```

11.4.3. Explicación del Código

1. **`quick_sort(arr)`**: Nuestra función toma un vector `arr`.

2. `if (n <= 1)`: El **caso base** de la recursión. Si el array tiene 0 o 1 elemento, se devuelve tal cual.
3. `pivot_index <- n` / `pivot <- arr[pivot_index]`: Elegimos el último elemento como pivote. Esta es una elección simple, pero puede llevar al peor caso si el array ya está ordenado o casi ordenado.
4. `remaining_elements <- arr[-pivot_index]`: Creamos un nuevo vector que contiene todos los elementos excepto el pivote.
5. `less <- remaining_elements[remaining_elements < pivot]`: Filtra los elementos de `remaining_elements` que son menores que el pivote.
6. `greater <- remaining_elements[remaining_elements >= pivot]`: Filtra los elementos que son mayores o iguales que el pivote. Es importante incluir los iguales para evitar bucles infinitos en casos con duplicados si solo se usara `>` y el pivote fuera único.
7. `return(c(quick_sort(less), pivot, quick_sort(greater)))`: Esta es la fase de combinación.
   - Llama recursivamente a `quick_sort` en la sublista `less`.
   - El `pivot` se coloca en medio, ya que está en su posición final.
   - Llama recursivamente a `quick_sort` en la sublista `greater`.
   - `c()` concatena estos tres resultados en el array final ordenado.

11.4.4. Complejidad

- **Tiempo**:
  - **Peor caso**: O(n^2). Ocurre cuando la elección del pivote es consistentemente mala (por ejemplo, siempre el elemento más pequeño o más grande), lo que resulta en particiones desequilibradas (una sublista de `n-1` elementos y otra de 0).
  - **Caso promedio**: O(n log n). Con una buena elección de pivote (que divide el array en dos subproblemas de tamaño similar), Quicksort es muy eficiente.
  - **Mejor caso**: O(n log n).
- **Espacio**:
  - **Caso promedio**: O(log n). Esto se debe al espacio utilizado por la pila de recursión para almacenar las llamadas a la función.
  - **Peor caso**: O(n). Si las particiones son muy desequilibradas, la pila de recursión puede crecer linealmente con el tamaño de `n`.
  - Nuestra implementación en R, al crear nuevos vectores `less` y `greater` en cada llamada, tiene una complejidad espacial mayor en la práctica que una implementación "in-place" típica de Quicksort, que solo usa espacio para la pila de recursión.

11.4.5. Ejemplo

Veamos `quick_sort` en acción.

```r
# Ejemplo 1: Un vector desordenado
vector_desordenado <- c(10, 80, 30, 90, 40, 50, 70)
print(paste("Vector original:", paste(vector_desordenado, collapse = ", ")))
```

```
[1] "Vector original: 10, 80, 30, 90, 40, 50, 70"
```

```r
vector_ordenado <- quick_sort(vector_desordenado)
print(paste("Vector ordenado (Quicksort):", paste(vector_ordenado, collapse = ", ")))
```

```
[1] "Vector ordenado (Quicksort): 10, 30, 40, 50, 70, 80, 90"
```

```r
# Ejemplo 2: Un vector con números negativos
vector_negativos <- c(-5, 3, -1, 8, 0, -10)
print(paste("Vector con negativos:", paste(vector_negativos, collapse = ", ")))
```

```
[1] "Vector con negativos: -5, 3, -1, 8, 0, -10"
```

```r
vector_ordenado_neg <- quick_sort(vector_negativos)
print(paste("Vector ordenado (Quicksort):", paste(vector_ordenado_neg, collapse = ", ")))
```

```
[1] "Vector ordenado (Quicksort): -10, -5, -1, 0, 3, 8"
```

11.5. Conclusión

Hemos recorrido cuatro algoritmos de ordenación fundamentales, cada uno con su enfoque único y características de rendimiento.

- **Bubble Sort** e **Insertion Sort** son algoritmos simples de entender e implementar, adecuados para arrays pequeños o casi ordenados (donde rinden en O(n)). Sin embargo, su complejidad de O(n^2) los hace imprácticos para grandes volúmenes de datos.
- **Mergesort** y **Quicksort** son algoritmos de "divide y vencerás" que ofrecen un rendimiento mucho mejor, generalmente O(n log n).
    - **Mergesort** es estable y tiene un rendimiento consistente de O(n log n) en todos los casos, pero requiere O(n) de espacio adicional.
    - **Quicksort** es típicamente el más rápido en la práctica (O(n log n) promedio), especialmente en implementaciones "in-place", aunque su peor caso es O(n^2) y puede ser inestable. La elección del pivote es crítica para su eficiencia.

En R, para tareas de ordenación cotidianas, generalmente se recomienda usar las funciones nativas optimizadas como `sort()` o `order()`, ya que están implementadas en C/Fortran y son extremadamente eficientes. Sin embargo, comprender los algoritmos subyacentes es crucial para cualquier científico de datos o programador que desee escribir código eficiente y comprender las bases de la informática. ¡Espero que este capítulo le haya proporcionado una base sólida!

12 Algoritmos de Búsqueda

¡Hola a todos y bienvenidos a este capítulo sobre algoritmos de búsqueda en R! Como analistas de datos, científicos de datos o simplemente entusiastas de la programación, constantemente nos enfrentamos a la tarea de encontrar elementos específicos dentro de colecciones de datos. Ya sea que estemos buscando una transacción particular en un enorme conjunto de datos financieros, un usuario específico en una base de datos de millones de registros, o un valor dentro de un vector en R, la eficiencia con la que realizamos estas búsquedas puede tener un impacto significativo en el rendimiento de nuestras aplicaciones y en nuestro tiempo de espera.

En este capítulo, exploraremos dos de los algoritmos de búsqueda más fundamentales y ampliamente utilizados: la Búsqueda Secuencial (también conocida como Búsqueda Lineal) y la Búsqueda Binaria. Entenderemos cómo funcionan, cuándo usar cada uno y cómo implementarlos eficientemente en R. ¡Prepárense para optimizar sus búsquedas!

12.1. Búsqueda Secuencial (Linear Search)

La búsqueda secuencial es el algoritmo de búsqueda más directo e intuitivo. Imaginen que están buscando un libro específico en una estantería desorganizada. Lo más lógico sería empezar por el primer libro, revisarlo, luego pasar al segundo, y así sucesivamente, hasta que encuentren el libro que buscan o hasta que se queden sin libros. Eso es precisamente lo que hace la búsqueda secuencial.

Este algoritmo examina cada elemento de la colección uno por uno, en el orden en que aparecen, hasta que encuentra el elemento deseado o hasta que ha revisado todos los elementos sin éxito.

12.1.1. ¿Cómo funciona?

1. Se inicia en el primer elemento de la colección.
2. Se compara el elemento actual con el valor objetivo que estamos buscando.
3. Si coinciden, la búsqueda termina y se devuelve la posición (índice) del elemento.
4. Si no coinciden, se avanza al siguiente elemento.
5. Este proceso se repite hasta que se encuentra el elemento o se alcanza el final de la colección.

12.1.2. Implementación en R

Vamos a implementar una función `busqueda_secuencial` en R. Esta función tomará un vector (`datos`) y un valor objetivo (`objetivo`) como entrada.

```
#' Función para realizar una búsqueda secuencial en un vector
#'
#' @param datos Un vector numérico o de caracteres donde buscar.
#' @param objetivo El valor que se desea encontrar.
#' @return El índice (posición) del objetivo si se encuentra, de lo contrario, NULL.
#'
#' @examples
#' mi_vector <- c(10, 5, 20, 15, 30, 25)
#' busqueda_secuencial(mi_vector, 15) # Debería devolver 4
#' busqueda_secuencial(mi_vector, 99) # Debería devolver NULL
busqueda_secuencial <- function(datos, objetivo) {
  # Recorremos el vector desde el primer elemento hasta el último
  for (i in seq_along(datos)) {
    # Comparamos el elemento actual con el objetivo
    if (datos[i] == objetivo) {
      # Si coinciden, devolvemos su índice (posición)
      message(paste("Objetivo", objetivo, "encontrado en el índice", i))
      return(i)
    }
  }
  # Si el bucle termina y no hemos encontrado el objetivo, devolvemos NULL
  message(paste("Objetivo", objetivo, "no encontrado en el vector."))
  return(NULL)
}
```

12.1.3. Ejemplos de Uso

Veamos nuestra función en acción con algunos ejemplos.

```
# Ejemplo 1: El objetivo está presente
numeros <- c(4, 2, 7, 1, 9, 5, 8, 3, 6)
busqueda_secuencial(numeros, 9)
```

```
Objetivo 9 encontrado en el índice 5
```

```
[1] 5
```

```r
# Ejemplo 2: El objetivo no está presente
busqueda_secuencial(numeros, 10)
```

```
Objetivo 10 no encontrado en el vector.

NULL
```

```r
# Ejemplo 3: Con datos de texto
frutas <- c("manzana", "platano", "cereza", "dátil", "elderberry")
busqueda_secuencial(frutas, "cereza")
```

```
Objetivo cereza encontrado en el índice 3

[1] 3
```

```r
busqueda_secuencial(frutas, "kiwi")
```

```
Objetivo kiwi no encontrado en el vector.

NULL
```

```r
# Ejemplo 4: El objetivo es el primer elemento
busqueda_secuencial(numeros, 4)
```

```
Objetivo 4 encontrado en el índice 1

[1] 1
```

```r
# Ejemplo 5: El objetivo es el último elemento
busqueda_secuencial(numeros, 6)
```

```
Objetivo 6 encontrado en el índice 9

[1] 9
```

12.1.4. Eficiencia y Casos de Uso

La principal ventaja de la búsqueda secuencial es su simplicidad y que no requiere que los datos estén ordenados. Sin embargo, su principal desventaja es su eficiencia.

- **Complejidad Temporal (Worst Case):** O(n). En el peor de los casos (cuando el elemento no está presente o está al final), el algoritmo debe revisar todos los `n` elementos.
- **Complejidad Temporal (Best Case):** O(1). En el mejor de los casos (cuando el elemento es el primero), solo necesita una comparación.
- **Complejidad Temporal (Average Case):** O(n). En promedio, revisará `n/2` elementos.

La búsqueda secuencial es apropiada para: * Conjuntos de datos pequeños. * Datos que no están ordenados y ordenar los datos sería más costoso que realizar la búsqueda secuencial. * Cuando solo se necesitan realizar unas pocas búsquedas.

Para conjuntos de datos grandes, la búsqueda secuencial puede ser muy lenta. Es aquí donde la búsqueda binaria brilla.

12.2. Búsqueda Binaria (Binary Search)

A diferencia de la búsqueda secuencial, la búsqueda binaria es un algoritmo mucho más eficiente, pero tiene un requisito fundamental: **los datos deben estar ordenados**.

Piensen en cómo buscan una palabra en un diccionario. No empiezan por la primera palabra y revisan una por una. Abren el diccionario por la mitad, miran si la palabra está antes o después, y luego descartan la mitad del diccionario que no les sirve. Repiten este proceso en la mitad restante hasta encontrar la palabra. La búsqueda binaria opera bajo el mismo principio de "divide y vencerás".

12.2.1. ¿Cómo funciona?

1. Asegurarse de que el vector esté **ordenado**. Si no lo está, primero deben ordenarlo.
2. Se define un rango de búsqueda dentro del vector, inicialmente todo el vector. Este rango está delimitado por un índice `inicio` y un índice `fin`.
3. Se calcula el índice del elemento central del rango: `medio = (inicio + fin) / 2`.
4. Se compara el elemento en la posición `medio` con el valor objetivo:
    - Si coinciden, el elemento ha sido encontrado y se devuelve `medio`.
    - Si el elemento en `medio` es **menor** que el objetivo, significa que el objetivo (si existe) debe estar en la mitad **derecha** del rango. Se ajusta `inicio` a `medio + 1`.
    - Si el elemento en `medio` es **mayor** que el objetivo, significa que el objetivo (si existe) debe estar en la mitad **izquierda** del rango. Se ajusta `fin` a `medio - 1`.

5. Los pasos 2 a 4 se repiten hasta que el objetivo se encuentra o hasta que el rango de búsqueda se reduce a cero (lo que significa que el objetivo no está presente).

12.2.2. Implementación en R

Implementaremos una versión iterativa de la búsqueda binaria.

```
#' Función para realizar una búsqueda binaria en un vector ordenado
#'
#' @param datos Un vector numérico o de caracteres ORDENADO donde buscar.
#' @param objetivo El valor que se desea encontrar.
#' @return El índice (posición) del objetivo si se encuentra, de lo contrario, NULL.
#'
#' @examples
#' vector_ordenado <- c(5, 10, 15, 20, 25, 30)
#' busqueda_binaria(vector_ordenado, 20) # Debería devolver 4
#' busqueda_binaria(vector_ordenado, 12) # Debería devolver NULL
busqueda_binaria <- function(datos, objetivo) {
  # Verificar que el vector no esté vacío
  if (length(datos) == 0) {
    message("El vector está vacío.")
    return(NULL)
  }

  # Se asume que el vector 'datos' ya está ordenado.
  # Si no estuviera ordenado, necesitaríamos una línea como:
  # datos <- sort(datos)
  # Pero esto añade una complejidad O(n log n) y no es parte del algoritmo de búsqueda binar

  inicio <- 1
  fin <- length(datos)

  while (inicio <= fin) {
    # Calcular el punto medio. Usamos floor para asegurar un índice entero.
    medio <- floor((inicio + fin) / 2)

    # Comparamos el elemento en el medio con el objetivo
    if (datos[medio] == objetivo) {
      message(paste("Objetivo", objetivo, "encontrado en el índice", medio))
      return(medio) # ¡Encontrado!
    } else if (datos[medio] < objetivo) {
      # El objetivo está en la mitad derecha
```

```r
      inicio <- medio + 1
    } else {
      # El objetivo está en la mitad izquierda
      fin <- medio - 1
    }
  }

  # Si el bucle termina, el objetivo no fue encontrado
  message(paste("Objetivo", objetivo, "no encontrado en el vector."))
  return(NULL)
}
```

12.2.3. Ejemplos de Uso

Recuerden, el vector DEBE estar ordenado para que la búsqueda binaria funcione correctamente.

```r
# Creamos un vector ordenado
numeros_ordenados <- c(1, 3, 5, 7, 9, 11, 13, 15, 17, 19)

# Ejemplo 1: El objetivo está presente
busqueda_binaria(numeros_ordenados, 9)
```

```
Objetivo 9 encontrado en el índice 5
```

```
[1] 5
```

```r
# Ejemplo 2: El objetivo no está presente
busqueda_binaria(numeros_ordenados, 10)
```

```
Objetivo 10 no encontrado en el vector.
```

```
NULL
```

```r
# Ejemplo 3: El objetivo es el primer elemento
busqueda_binaria(numeros_ordenados, 1)
```

```
Objetivo 1 encontrado en el índice 1
```

```
[1] 1
```

```r
# Ejemplo 4: El objetivo es el último elemento
busqueda_binaria(numeros_ordenados, 19)
```

```
Objetivo 19 encontrado en el índice 10
```

```
[1] 10
```

```r
# Ejemplo 5: Con un vector de un solo elemento (objetivo presente)
busqueda_binaria(c(5), 5)
```

```
Objetivo 5 encontrado en el índice 1
```

```
[1] 1
```

```r
# Ejemplo 6: Con un vector de un solo elemento (objetivo ausente)
busqueda_binaria(c(5), 10)
```

```
Objetivo 10 no encontrado en el vector.
```

```
NULL
```

¡Advertencia! ¿Qué pasa si el vector no está ordenado?

```r
numeros_desordenados <- c(5, 1, 9, 3, 7)
# Si intentamos buscar 3, ¡dará un resultado incorrecto o NULL!
# (En este caso, 3 está en la posición 4, pero la búsqueda binaria podría devolver NULL o alg
busqueda_binaria(numeros_desordenados, 3) # Resultado erróneo porque el vector no está ordena
```

```
Objetivo 3 no encontrado en el vector.
```

```
NULL
```

Para corregirlo, simplemente ordenamos el vector antes de buscar:

```r
numeros_desordenados_ordenados <- sort(numeros_desordenados)
busqueda_binaria(numeros_desordenados_ordenados, 3) # Ahora sí funciona correctamente
```

```
Objetivo 3 encontrado en el índice 2
```

```
[1] 2
```

12.2.4. Eficiencia y Casos de Uso

La eficiencia de la búsqueda binaria es su mayor fortaleza. En cada paso, el algoritmo reduce el espacio de búsqueda a la mitad.

- **Complejidad Temporal:** O(log n). Esto significa que el tiempo de ejecución crece logarítmicamente con el tamaño de los datos. Para un vector de 1 millón de elementos, la búsqueda binaria podría necesitar solo unas 20 comparaciones (log de 1,000,000 es aproximadamente 19.9), mientras que la búsqueda secuencial podría necesitar hasta 1 millón.

La búsqueda binaria es ideal para: * Grandes conjuntos de datos. * Cuando los datos están (o pueden estar) ordenados. * Cuando se realizan muchas búsquedas en el mismo conjunto de datos (amortizando el costo de ordenamiento inicial).

12.3. Comparación y Cuándo Usar Cada Uno

Característica	Búsqueda Secuencial (Lineal)	Búsqueda Binaria
Requisito de Orden	No requiere	**Requiere**
Complejidad Temporal	O(n) (lineal)	O(log n) (logarítmica)
Tamaño de Datos	Pequeños	Grandes
Simplicidad	Muy simple de implementar	Más compleja
Escenarios Típicos	Datos desordenados, pocas búsquedas, listas pequeñas	Datos ordenados, muchas búsquedas, listas grandes

En resumen:

- Si tus datos no están ordenados y el tamaño es pequeño, o si solo necesitas hacer una única búsqueda y el costo de ordenar sería excesivo, usa la **Búsqueda Secuencial**.

- Si tus datos están ordenados o si necesitas realizar muchas búsquedas en un conjunto de datos grande (haciendo que el costo de ordenar los datos inicialmente valga la pena), la **Búsqueda Binaria** es la elección superior por su eficiencia.

En R, para tareas comunes de búsqueda, funciones como `which()` o `%in%` a menudo son eficientes para vectores, ya que están implementadas en C y optimizadas. Sin embargo, entender los algoritmos subyacentes como la búsqueda secuencial y binaria es crucial para comprender la eficiencia y el diseño de soluciones en contextos más complejos, como la implementación en estructuras de datos personalizadas o en escenarios de entrevistas técnicas.

12.4. Conclusión

Hemos cubierto los fundamentos de la búsqueda secuencial y la búsqueda binaria. Ahora entienden cómo funcionan estos algoritmos, cómo implementarlos en R y, lo que es más importante, cuándo y por qué elegir uno sobre el otro. La eficiencia de sus programas a menudo depende de estas decisiones algorítmicas básicas, y dominar estos conceptos es un paso crucial para convertirse en un experto en R y en programación en general.

¡Sigan practicando y experimentando con estos algoritmos! La próxima vez que necesiten encontrar algo en un conjunto de datos, podrán tomar una decisión informada sobre la mejor estrategia de búsqueda. ¡Hasta el próximo capítulo!

13 Estructuras No Lineales

¡Hola a todos y bienvenidos a este capítulo sobre Estructuras No Lineales en R! En el mundo de la programación y el análisis de datos, no todo es una tabla bonita o un vector secuencial. A menudo, la información que manejamos tiene relaciones complejas, jerarquías o interconexiones que las estructuras de datos lineales simplemente no pueden representar de manera eficiente o intuitiva. Aquí es donde entran en juego los grafos y los árboles binarios.

En este capítulo, exploraremos cómo R nos permite trabajar con estas estructuras fundamentales, dotándonos de las herramientas para modelar y analizar datos con relaciones más intrincadas. Prepárense para ver cómo R, con la ayuda de paquetes poderosos, transforma estos conceptos abstractos en herramientas prácticas para la ciencia de datos.

13.1. Grafos: Tejiendo Redes de Relaciones

Un **grafo** es una de las estructuras de datos no lineales más potentes y versátiles. Imaginen una red social, las conexiones de una red eléctrica, el mapa de carreteras de una ciudad o incluso las interacciones entre proteínas en un organismo. Todos estos pueden ser modelados como grafos.

Formalmente, un grafo se compone de un conjunto de **vértices** (también llamados nodos) y un conjunto de **aristas** (también llamadas enlaces o conexiones) que unen pares de vértices.

13.1.1. Tipos de Grafos

- **Dirigidos (Directed)**: Las aristas tienen una dirección específica (ej. un seguimiento en Twitter: A sigue a B, pero B no necesariamente sigue a A).
- **No Dirigidos (Undirected)**: Las aristas no tienen una dirección (ej. una amistad en Facebook: si A es amigo de B, B es amigo de A).
- **Ponderados (Weighted)**: Las aristas tienen un valor o "peso" asociado (ej. la distancia entre dos ciudades, la fuerza de una conexión).
- **No Ponderados (Unweighted)**: Las aristas simplemente indican una conexión, sin un valor adicional.

13.1.2. Grafos en R: El Paquete `igraph`

En R, el paquete por excelencia para trabajar con grafos es `igraph`. Es una biblioteca extremadamente eficiente y rica en funcionalidades para la creación, manipulación y análisis de redes complejas.

Primero, si no lo tienen instalado, es momento de hacerlo:

```
# Instalar igraph si no lo tienen
# install.packages("igraph")
```

Ahora, carguemos el paquete y empecemos a construir nuestro primer grafo.

```
library(igraph)

# Creamos un grafo simple no dirigido
# Podemos definirlo a partir de un listado de aristas
g1 <- graph_from_literal(A-B, B-C, C-D, D-A, B-D, A-C)
print(g1)
```

Como pueden ver en la salida, `g1` es un grafo `UN--` (Undirected, Named) con 4 vértices y 6 aristas.

Podemos visualizarlo fácilmente:

```
# Visualizar el grafo
plot(g1)
```

> **Nota**
>
> `plot()` para `igraph` es muy versátil. Permite personalizar el diseño, colores, tamaños y muchas otras propiedades visuales. Experimenten con sus argumentos como `vertex.color`, `edge.color`, `layout`, etc.

También podemos crear grafos a partir de otros formatos, como una matriz de adyacencia (qué vértices están conectados directamente) o una lista de aristas.

```
# Grafo a partir de una matriz de adyacencia
adj_matrix <- matrix(c(0, 1, 1, 0,
                       1, 0, 1, 1,
                       1, 1, 0, 1,
                       0, 1, 1, 0), nrow = 4, byrow = TRUE)
```

```r
g2 <- graph_from_adjacency_matrix(adj_matrix, mode = "undirected")
V(g2)$name <- c("V1", "V2", "V3", "V4") # Asignar nombres a los vértices
plot(g2, main = "Grafo desde Matriz de Adyacencia")
```

```r
# Grafo dirigido a partir de una lista de aristas
edge_list <- data.frame(
  from = c("Alice", "Bob", "Charlie", "David", "Eve", "Alice", "Charlie"),
  to = c("Bob", "Charlie", "David", "Eve", "Alice", "Charlie", "Eve")
)
g3 <- graph_from_data_frame(edge_list, directed = TRUE)
plot(g3, main = "Grafo Dirigido de Red Social", layout = layout_nicely(g3))
```

13.1.3. Propiedades y Análisis de Grafos

`igraph` nos permite calcular una gran cantidad de métricas y propiedades de un grafo:

```r
# Número de vértices y aristas
vcount(g3)
ecount(g3)

# Grado de los vértices (número de conexiones)
# Para grafos dirigidos, podemos ver in-degree (entrantes) y out-degree (salientes)
degree(g3, mode = "in")
degree(g3, mode = "out")
degree(g3, mode = "all") # Suma de in y out

# Componentes conectados (subgrafos donde todos los vértices están conectados entre sí)
components(g1)

# Camino más corto entre dos vértices
shortest_paths(g3, from = "Alice", to = "Eve", output = "vpath")$vpath[[1]]

# Centralidad de un vértice (importancia en la red)
# Centralidad de intermediación (betweenness centrality)
# ¿Cuántos caminos más cortos pasan a través de este vértice?
betweenness(g3, normalized = TRUE)

# Centralidad de cercanía (closeness centrality)
# ¿Qué tan cerca está un vértice de todos los demás?
closeness(g3, normalized = TRUE)
```

La exploración de grafos es un campo vasto y fascinante. `igraph` es una herramienta indispensable para cualquier científico de datos que trabaje con relaciones y redes.

13.2. Árboles Binarios: Jerarquías con Ramificaciones Limitadas

Un **árbol** es un tipo especial de grafo no dirigido, acíclico y conectado. Esto significa que no tiene ciclos y cada nodo está conectado a todos los demás de alguna manera. En un árbol, generalmente se designa un nodo como la **raíz**, y a partir de ahí, los nodos se organizan jerárquicamente en **padres**, **hijos** y **hojas** (nodos sin hijos).

Un **árbol binario** es un tipo específico de árbol donde cada nodo tiene como máximo dos hijos: un hijo "izquierdo" y un hijo "derecho". Son estructuras fundamentales en informática para organizar datos de manera eficiente para búsqueda, inserción y eliminación. Un caso particular muy útil es el **Árbol Binario de Búsqueda (BST)**, donde para cada nodo, todos los valores en su subárbol izquierdo son menores que su propio valor, y todos los valores en su subárbol derecho son mayores.

13.2.1. Implementando Árboles Binarios en R

A diferencia de los grafos, donde `igraph` es el estándar, no existe un paquete único y universalmente adoptado en R para la creación de estructuras de árboles binarios genéricas como tipos de datos abstractos (al margen de implementaciones específicas para algoritmos como árboles de decisión). Sin embargo, podemos construir nuestra propia representación usando los tipos de datos de R, como listas o clases S3/S4, para ilustrar el concepto.

Aquí construiremos una representación sencilla de un nodo de un árbol binario utilizando una clase S3, y mostraremos cómo se conectarían para formar un árbol.

```r
# Definición de un nodo de árbol binario como una clase S3
# Un nodo tendrá un valor, y punteros a sus hijos izquierdo y derecho
# (que a su vez serán otros nodos)
make_node <- function(value, left = NULL, right = NULL) {
  node <- list(value = value, left = left, right = right)
  class(node) <- "binary_node"
  return(node)
}

# Función para imprimir un nodo (método para la clase S3)
print.binary_node <- function(x, ...) {
  cat("Node Value:", x$value, "\n")
  cat("  Left Child:", if (!is.null(x$left)) x$left$value else "NULL", "\n")
```

```r
  cat("  Right Child:", if (!is.null(x$right)) x$right$value else "NULL", "\n")
}

# Construyamos un pequeño árbol binario
# Nivel 3 (hojas)
node4 <- make_node(4)
node6 <- make_node(6)
node9 <- make_node(9)
node12 <- make_node(12)

# Nivel 2
node5 <- make_node(5, left = node4, right = node6)
node10 <- make_node(10, left = node9, right = node12)

# Nivel 1 (raíz)
root_node <- make_node(8, left = node5, right = node10)

# Mostramos el nodo raíz y sus hijos
print(root_node)
print(root_node$left)
print(root_node$right)
print(root_node$left$left) # El hijo izquierdo del hijo izquierdo de la raíz
```

Este ejemplo demuestra cómo cada nodo es una entidad que contiene su propio valor y referencias a sus hijos. Al manipular estas referencias, podemos construir y modificar la estructura del árbol.

13.2.2. Operaciones Comunes en Árboles Binarios (Conceptos)

- **Inserción**: Añadir un nuevo nodo al árbol, manteniendo sus propiedades (ej. en un BST, manteniendo el orden).
- **Búsqueda**: Encontrar un nodo con un valor específico.
- **Eliminación**: Quitar un nodo del árbol.
- **Recorridos (Traversals)**: Visitar todos los nodos del árbol de una manera sistemática.
    - **In-order**: Izquierda -> Raíz -> Derecha (útil para BSTs, devuelve los elementos ordenados).
    - **Pre-order**: Raíz -> Izquierda -> Derecha (útil para copiar o serializar el árbol).
    - **Post-order**: Izquierda -> Derecha -> Raíz (útil para eliminar el árbol o evaluar expresiones).

Implementar estas operaciones completamente en R como funciones recursivas es posible y una excelente práctica para entender cómo funcionan estas estructuras. Sin embargo, para mantener este capítulo conciso, nos centraremos en el concepto.

> **Tip**
>
> Para una implementación más robusta de árboles de propósito general en R, consideren explorar el paquete `data.tree`. Aunque no se limita a árboles binarios, proporciona una estructura poderosa para trabajar con datos jerárquicos de manera intuitiva.

13.3. Conclusión

Hemos recorrido un camino fascinante por el mundo de las estructuras no lineales en R. Los **grafos**, con el soporte excepcional del paquete `igraph`, nos permiten modelar y analizar sistemas complejos donde las relaciones son primordiales, desde redes sociales hasta interacciones biológicas. Los **árboles binarios**, aunque su implementación en R a nivel de estructura de datos pura puede requerir un enfoque más "manual" o específico, son fundamentales para entender cómo se organizan y procesan los datos de forma jerárquica, subyaciendo a muchos algoritmos de búsqueda y toma de decisiones.

Dominar estas estructuras amplía enormemente su capacidad para abordar problemas de datos más complejos y estructurados. Sigan experimentando con `igraph` y, si se sienten aventureros, intenten implementar algoritmos básicos de árboles binarios en R para solidificar su comprensión.

¡Hasta la próxima!

14 Estructuras Avanzadas

¡Bienvenidos a un viaje por el fascinante mundo de las estructuras de datos avanzadas! Como expertos en R, sabemos que si bien el lenguaje brilla por su capacidad de manipulación de datos a alto nivel y su ecosistema de paquetes estadísticos, comprender las estructuras subyacentes puede ser crucial para optimizar algoritmos o entender cómo funcionan ciertas herramientas internas o externas a R. En este capítulo, exploraremos tres estructuras de datos sofisticadas: los Árboles AVL, los Árboles B y los Fibonacci Heaps. Nos centraremos en sus fundamentos, sus casos de uso y cómo podemos pensar en ellas o interactuar con ellas desde la perspectiva de R.

14.1. Árboles AVL

Los árboles AVL (por sus inventores Adelson-Velsky y Landis) son un tipo de árbol de búsqueda binario auto-balanceado. Esto significa que garantizan que la altura del árbol sea logarítmica con respecto al número de nodos, lo que a su vez asegura que las operaciones de búsqueda, inserción y eliminación se realicen en tiempo $O(\log n)$. La clave de su auto-balanceo reside en mantener el "factor de balance" de cada nodo (la diferencia de altura entre sus subárboles izquierdo y derecho) entre -1, 0 y 1. Si esta condición se rompe tras una inserción o eliminación, el árbol realiza una serie de rotaciones (simples o dobles, a la izquierda o a la derecha) para restaurar el balance.

14.1.1. ¿Por qué son importantes?

- **Rendimiento garantizado:** A diferencia de un árbol de búsqueda binario simple, que puede degenerar en una lista enlazada (y $O(n)$ en el peor caso), los árboles AVL siempre ofrecen un rendimiento $O(\log n)$.
- **Aplicaciones:** Son útiles en bases de datos para indexación, sistemas de archivos y cualquier aplicación donde se necesite un diccionario o conjunto ordenado con inserciones y eliminaciones frecuentes y eficientes.

14.1.2. Árboles AVL y R

En R, la implementación de un árbol AVL desde cero es una tarea no trivial y, sinceramente, rara vez el enfoque más eficiente para la mayoría de los problemas que abordamos. R sobresale en operaciones vectorizadas y en el manejo de grandes volúmenes de datos mediante estructuras como `data.frame`, `data.table` o `tibble`, que están altamente optimizadas y, a menudo, respaldadas por código C/C++.

Sin embargo, podemos ilustrar los conceptos básicos de un nodo y cómo podríamos pensar en la estructura de un árbol.

```r
# Definición conceptual de un nodo para un árbol binario
# En R, esto sería típicamente una lista recursiva.
create_node <- function(value) {
  list(
    value = value,
    left = NULL,
    right = NULL,
    height = 1, # Altura del nodo (1 para nodo hoja)
    balance_factor = 0 # Diferencia de altura entre subárboles
  )
}

# Creamos algunos nodos para ilustrar
root <- create_node(10)
root$left <- create_node(5)
root$right <- create_node(15)
root$left$right <- create_node(7)

# Visualización simple (conceptual)
str(root)
```

Una implementación completa de rotaciones y rebalanceo en R sería bastante extensa. Para escenarios donde se requiere un diccionario o conjunto ordenado, un enfoque más idiomático en R sería:

1. **Tablas hash (environments o `list` con nombres):** Para búsquedas $O(1)$ promedio, pero sin orden inherente.
2. **Vectores ordenados con `match` o `findInterval`:** Para conjuntos pequeños a medianos, las operaciones vectorizadas de R son increíblemente rápidas.
3. **Paquetes especializados:** El paquete `collections` ofrece una clase `Deque` o `PriorityQueue` que puede ser útil para ciertos patrones, aunque no implementa directamente árboles AVL.

```r
# Ejemplo de un "set" ordenado conceptualmente usando un vector
sorted_vector <- sort(c(10, 5, 15, 7, 20, 3))
print(sorted_vector)

# Búsqueda eficiente en un vector ordenado
# Equivale a una búsqueda binaria en términos de complejidad O(log n)
# pero es una función de C optimizada en R.
which(sorted_vector == 7)
findInterval(7, sorted_vector) # Indica el índice donde se insertaría o se encuentra
```

En resumen, mientras que los árboles AVL son fundamentales en la ciencia de la computación, en R, su implementación directa rara vez es el camino más eficiente. En su lugar, nos apoyamos en las optimizaciones internas de R para vectores ordenados o utilizamos estructuras de datos proporcionadas por paquetes bien diseñados que pueden tener sus propias implementaciones en C/C++.

14.2. Árboles B

Los árboles B son estructuras de datos de árbol que generalizan el concepto de árbol binario de búsqueda, permitiendo que un nodo tenga más de dos hijos. Fueron diseñados específicamente para optimizar el acceso a datos en sistemas de almacenamiento secundario (como discos duros), donde los accesos son lentos y costosos.

14.2.1. Características Clave

- **Multi-camino:** Cada nodo puede tener un gran número de claves y un número aún mayor de hijos (orden `m`).
- **Balanceado:** Al igual que los AVL, los árboles B están balanceados, lo que significa que todos los caminos desde la raíz hasta las hojas tienen la misma longitud. Esto garantiza un rendimiento $O(\log_m n)$ para las operaciones de búsqueda, inserción y eliminación, donde `m` es el orden del árbol.
- **Optimizado para disco:** El tamaño de un nodo en un árbol B se elige para que coincida con el tamaño de un bloque de disco (o un múltiplo de él). Esto minimiza el número de accesos a disco, ya que al leer un nodo, se lee un bloque completo de datos.

14.2.2. ¿Por qué son importantes?

- **Bases de datos:** Son la estructura de datos fundamental detrás de la indexación en la mayoría de los sistemas de gestión de bases de datos (SQL, NoSQL). Un índice B-tree permite que las consultas accedan a filas específicas de manera extremadamente rápida.

- **Sistemas de archivos:** Algunos sistemas de archivos utilizan variantes de árboles B (como los B+-trees) para organizar los metadatos y los bloques de datos.

14.2.3. Árboles B y R

R, por su naturaleza, no implementa árboles B internamente para la gestión de sus objetos principales. Sin embargo, como analistas de datos, interactuamos constantemente con sistemas que *sí* los utilizan. Cada vez que ejecutamos una consulta SQL con un `WHERE` o `ORDER BY` en una columna indexada, estamos indirectamente aprovechando el poder de un árbol B.

Podemos ilustrar el concepto de un nodo B-tree muy simplificadamente en R para entender cómo podría almacenar múltiples claves y punteros.

```
# Representación conceptual de un nodo B-tree
# Un nodo contiene varias claves y punteros a sus hijos.
# El número de claves es m-1, el número de hijos es m.
create_b_tree_node <- function(keys, children_pointers = NULL, is_leaf = FALSE) {
  list(
    keys = sort(keys), # Las claves están siempre ordenadas
    children = children_pointers, # Lista de punteros a nodos hijos
    is_leaf = is_leaf
  )
}

# Ejemplo de un nodo B-tree de orden 3 (máximo 2 claves, 3 hijos)
# Nodo raíz con dos claves
root_b_node <- create_b_tree_node(c(50, 100))

# Nodos hijos (conceptualmente, aquí solo representamos su "idea")
# En una implementación real, 'children' contendría referencias a otros nodos.
child1 <- create_b_tree_node(c(20, 40), is_leaf = TRUE)
child2 <- create_b_tree_node(c(60, 80), is_leaf = TRUE)
child3 <- create_b_tree_node(c(110, 130), is_leaf = TRUE)

root_b_node$children <- list(child1, child2, child3)

str(root_b_node, max.level = 2) # Limitar la profundidad para la visualización
```

Cuando trabajamos con R y grandes volúmenes de datos que residen en bases de datos, los paquetes como `DBI`, `RPostgres`, `RSQLite`, `odbc`, etc., nos permiten interactuar con estos sistemas que hacen un uso extensivo de árboles B.

```r
# Ejemplo conceptual: creando un índice en una base de datos usando R
library(DBI)
library(RSQLite)

# Conexión a una base de datos en memoria (SQLite)
con <- dbConnect(RSQLite::SQLite(), ":memory:")

# Crear una tabla
dbExecute(con, "CREATE TABLE usuarios (id INTEGER PRIMARY KEY, nombre TEXT, edad INTEGER)")

# Insertar algunos datos
dbAppendTable(con, "usuarios", data.frame(id = 1:5, nombre = paste0("Usuario", 1:5), edad = c

# Crear un índice en la columna 'edad'
# Detrás de escena, la base de datos probablemente usa un B-tree para este índice.
dbExecute(con, "CREATE INDEX idx_edad ON usuarios (edad)")

# Consulta que se beneficiará del índice
res <- dbGetQuery(con, "SELECT * FROM usuarios WHERE edad > 25 ORDER BY edad")
print(res)

# Desconectar
dbDisconnect(con)
```

Los árboles B son fundamentales para la persistencia y recuperación eficiente de datos a gran escala, y aunque R no los maneje directamente como objetos de primera clase, son una parte invisible pero vital de nuestro ecosistema de trabajo con datos.

14.3. Fibonacci Heap

Los Fibonacci Heaps son una estructura de datos de cola de prioridad (priority queue) compleja, conocida por sus excepcionales tiempos amortizados para un conjunto específico de operaciones, especialmente `decrease-key`. Fueron introducidos por Fredman y Tarjan en 1987.

14.3.1. Operaciones y Complejidad Amortizada

Las operaciones más comunes en un Fibonacci Heap y sus complejidades temporales (amortizadas):

- `make-heap`: $O(1)$

- `insert`: $O(1)$
- `minimum`: $O(1)$
- `extract-min`: $O(\log n)$
- `decrease-key`: $O(1)$
- `delete`: $O(\log n)$
- `union`: $O(1)$

La eficiencia de $O(1)$ amortizado para `decrease-key` es particularmente notoria, ya que en un heap binario esta operación suele ser $O(\log n)$.

14.3.2. ¿Por qué son importantes?

- **Algoritmos de grafos:** Su principal aplicación es la optimización de algoritmos de grafos como el algoritmo de Dijkstra para encontrar la ruta más corta en un grafo con pesos de arista no negativos, y el algoritmo de Prim para encontrar un árbol de expansión mínima. La reducción de la complejidad de `decrease-key` a $O(1)$ amortizado puede acelerar significativamente estos algoritmos en grafos densos.
- **Eficiencia teórica:** Son importantes en la teoría de la complejidad computacional por sus límites asintóticos.

14.3.3. Fibonacci Heaps y R

Al igual que con los árboles AVL y B, los Fibonacci Heaps no forman parte del conjunto de estructuras de datos básicas de R y su implementación desde cero sería extremadamente compleja y difícil de mantener. La razón principal es que R no está diseñado para la manipulación a bajo nivel de punteros y estructuras de datos dinámicas con la misma eficiencia que lenguajes como C o C++.

Para la mayoría de las necesidades de colas de prioridad en R, un heap binario es más que suficiente y mucho más simple de implementar o encontrar en paquetes existentes. El paquete `collections` proporciona una implementación eficiente de `PriorityQueue` que, bajo el capó, probablemente utiliza un heap binario o una estructura similar.

```r
# Ejemplo de uso de una cola de prioridad en R (usando el paquete 'collections')
# Primero, asegúrate de tener el paquete instalado: install.packages("collections")
library(collections)

# Crear una cola de prioridad
pq <- priority_queue()

# Insertar elementos con su prioridad (por defecto, valores menores tienen mayor prioridad)
pq$push("Tarea A", 3)
```

```r
pq$push("Tarea B", 1)
pq$push("Tarea C", 5)
pq$push("Tarea D", 2)

# Ver el elemento con la prioridad más alta (mínimo)
print(pq$front())

# Extraer el elemento con la prioridad más alta
extracted_item <- pq$pop()
print(extracted_item)
print(pq$front()) # El siguiente elemento con mayor prioridad

# Una operación de "decrease-key" no es directa en un heap binario típico,
# pero se podría simular eliminando y reinsertando, o creando una nueva entrada
# con la prioridad actualizada.

# Si 'Tarea A' ahora tiene prioridad 0 (más alta)
pq$push("Tarea A_new_priority", 0) # Esto crea una nueva entrada, no actualiza la existente
print(pq$pop()) # "Tarea A_new_priority" ahora será la siguiente en salir
```

Para los escenarios donde la eficiencia teórica de `decrease-key` es crítica (principalmente en implementaciones de algoritmos de grafos muy específicos para investigación o benchmarks), uno debería considerar implementar el algoritmo en un lenguaje de propósito general más cercano al hardware (como C++), o buscar bibliotecas especializadas que ya los hayan implementado. Para la mayoría de los análisis de datos y tareas estadísticas en R, la simplicidad y el buen rendimiento práctico de los heaps binarios son preferibles.

14.3.4. Conclusión

Hemos explorado tres estructuras de datos avanzadas: Árboles AVL, Árboles B y Fibonacci Heaps. Si bien sus implementaciones directas en R no son comunes debido a la naturaleza de alto nivel del lenguaje, comprender sus principios es crucial para cualquier experto en R:

- **Árboles AVL** nos enseñan la importancia del balance para el rendimiento garantizado en estructuras de búsqueda. En R, sus principios se ven reflejados en la eficiencia de las operaciones vectorizadas en datos ordenados.
- **Árboles B** son la columna vertebral de la persistencia de datos a gran escala, optimizando los accesos a disco. Como usuarios de R, nos beneficiamos de ellos cada vez que interactuamos con bases de datos relacionales.
- **Fibonacci Heaps** representan la cúspide de la optimización en colas de prioridad para escenarios específicos de algoritmos de grafos, aunque para la mayoría de las aplicaciones en R, las implementaciones de heaps binarios son más prácticas y eficientes.

Como expertos en R, nuestro poder no reside solo en codificar estructuras de datos desde cero, sino en saber cuándo y dónde se utilizan, cómo interactuar con ellas a través de las abstracciones del lenguaje, y cómo elegir la herramienta adecuada para cada problema, ya sea una función base de R, un paquete externo o la integración con sistemas externos.

15 Arboles_B_R

References